Generalized Disjunctive Programming with Derivative-Free Configuration Search

Albert Joon Lee [1], Woosik Kim [2], and David E. Bernal Neira [1,*]

[1]Davidson School of Chemical Engineering, Purdue University, West Lafayette, IN 47907, USA

[2]Elmore Family School of Electrical and Computer Engineering, Purdue University, West Lafayette, IN 47907, USA

*Email: dbernaln@purdue.edu

Abstract

Generalized disjunctive programming (GDP) describes process design alternatives with Boolean variables, disjunctions, and logical propositions, and fixing one configuration leaves a subproblem in the continuous variables. Conventional GDP methods require explicit algebraic formulations that simulators may not expose. Even when such formulations are available, nonconvex models can be intractable. The external variables define an integer lattice that derivative-free optimization (DFO) searches using subproblem values. We compare surrogate DFO methods and the direct-search method `NOMAD` with discrete steepest descent (`D-SDA`), logic-based solvers, mixed-integer nonlinear programming (MINLP) solvers, and enumeration. The benchmarks include nine process models, 21 test functions, a pharmaceutical flowsheet simulated in `PharmaPy`, and a catalytic distillation column with 100,947 configurations. On the largest reactor network, the surrogate methods `MISO` and `RBFOpt` reach the reference in nine or ten of ten runs with about 30 subproblem solves, whereas logic-based methods solve nearly one subproblem for each admissible configuration and most `D-SDA` descents end at one local minimum. In the flowsheet example, where GDP solvers do not apply, `RBFOpt` and `MISO` find the best of 36 simulated configurations in every run with 20 evaluations. For the catalytic column, `D-SDA` and `MISO` reach the best known design with 100 evaluations, whereas MINLP solvers find no design within 15% of it and global solvers return contradictory certificates. This search pays off when the configuration space is too large to enumerate, has deceptive local minima, or has MINLP reformulations that fail, and enumeration or a direct reformulation is simpler for small algebraic models.

Keywords: Generalized disjunctive programming; Black-box optimization; Derivative-free optimization; Surrogate modeling; Process systems engineering; Computational benchmarking.

1 Introduction

In chemical process design, evaluating a single candidate configuration can require costly optimization or simulation, making the choice of which configuration to explore next a central computational challenge. Process design involves discrete decisions, such as equipment selection, the number of processing stages, and operating modes, along with continuous variables such as flow rates, temperatures, and equipment sizes. Generalized disjunctive programming (GDP) provides a framework to represent these decisions through Boolean variables, logical propositions, and conditional constraints (Raman and Grossmann, 1994). In logic-based GDP solution methods, fixing the Boolean variables associated with a selected configuration determines which constraints remain active and yields a reduced optimization subproblem. When all discrete decisions are fixed, this subproblem can be formulated as a nonlinear programming (NLP) problem. Exploring multiple configurations may therefore require repeatedly solving NLP subproblems, which can be computationally

expensive. Consequently, identifying high-quality configurations with fewer evaluations of NLP subproblems is an important objective in solving GDP problems.

Conventional GDP solution methods generally rely on mixed-integer nonlinear programming (MINLP) reformulations or logic-based decomposition. The Big-M and Hull reformulations transform disjunctive constraints into algebraic formulations (Grossmann and Lee, 2003; Raman and Grossmann, 1994), while logic-based approaches, such as logic-based outer approximation (LOA) and the relaxation with integer cuts (RIC) of `GDPopt`, select discrete configurations and solve the corresponding reduced subproblems (Bergamini et al., 2005; Pyomo Development Team, n.d.; Türkay and Grossmann, 1996). In logic-based approaches, the strategy used to select discrete configurations can significantly affect the number of subproblem evaluations required. Both families need the algebraic equations of every disjunct. When a unit is described by a process simulator, these equations are not available, and when the model is nonconvex, a reformulation keeps the equations of units that a configuration does not contain, whose terms can be poorly scaled or undefined at the values that absent units take.

The Discrete-Steepest Descent Algorithm (`D-SDA`) was previously applied to process superstructure optimization by Liñán et al. (2020a). Building on this approach, our previous work introduced the Logic-Based Discrete-Steepest Descent Algorithm (`LD-SDA`), which represents ordered Boolean decisions using external integer variables (Ovalle et al., 2025). With this external-variable reformulation, the GDP becomes an optimization over an integer lattice whose objective is the value of the subproblem of each configuration. The GDP model still describes the alternatives, their logic, and the subproblem of each configuration, but the method that searches the lattice needs only the value of each configuration. `LD-SDA` explores the integer space through neighborhood search and discrete line search, evaluating selected configurations by solving their corresponding reduced optimization subproblems. However, as a local search method, `LD-SDA` may converge to different locally optimal solutions depending on the starting point and the selected neighborhood (Peng et al., 2024).

Because the lattice search needs only configuration values, any derivative-free optimization (DFO) method that handles integer variables can perform it, and the GDP model can be solved even when its subproblems are simulations or its MINLP reformulations fail. We pair GDP models in their external integer representation with DFO methods that search the configurations. Figure 1 summarizes this approach and the conventional GDP methods with which we compare it. We include the mesh adaptive direct-search solver `NOMAD` (Audet et al., 2022) and four surrogate-based methods, which use accumulated objective observations to guide new evaluations (Larson et al., 2019). `RBFOpt` builds surrogate models from radial basis functions and balances predicted objective values with exploration (Costa and Nannicini, 2018), and `MISO` combines radial-basis-function surrogate models with sampling strategies designed for integer variables (Müller, 2016). `NOMAD` and `MISO` performed best on mixed-integer problems in the comparison of Ploskas and Sahinidis (2022). The other two are Branch-and-Model (`BAM`) (Ma et al., 2023) and a variant of the factorization machine with quantum annealing (`FMQA`) framework (Kitai et al., 2020), in which classical simulated annealing replaces quantum annealing; we denote this variant `FMQA-SA`. `FMQA` represents its surrogate as a quadratic unconstrained binary optimization problem, the input format of quantum annealers, which makes it a candidate for a possibly quantum-assisted configuration search. We ask three questions. First, do these methods find high-quality configurations with fewer NLP subproblem solves than `D-SDA` and conventional GDP methods, and when does this advantage translate into shorter computing time? Second, does the same search solve GDP models whose subproblems are evaluated by a simulator? Third, does it solve GDP models whose MINLP reformulations defeat conventional solvers?

Candidate selection has its own cost, and evaluation costs vary between configurations (Moré and Wild, 2009). We therefore report three measures for every method: how often it reaches the reference objective, how many NLP subproblems it solves, and its estimated completion time. The complete enumeration of each configuration domain and uniform sampling serve as references, and all methods share initial configurations, evaluation budgets, and the responses of each configuration. The contributions are as follows:

- A use of the external-variable reformulation as the interface between a GDP model and a derivative-free configuration search, extended to subproblems defined by a process simulator and demonstrated on a lomustine manufacturing flowsheet simulated in `PharmaPy` and written as a GDP.

- A comparison of surrogate-based and direct-search configuration methods with local descent, logic-based and direct GDP methods, and full enumeration on four process GDP families and 21 Grid test

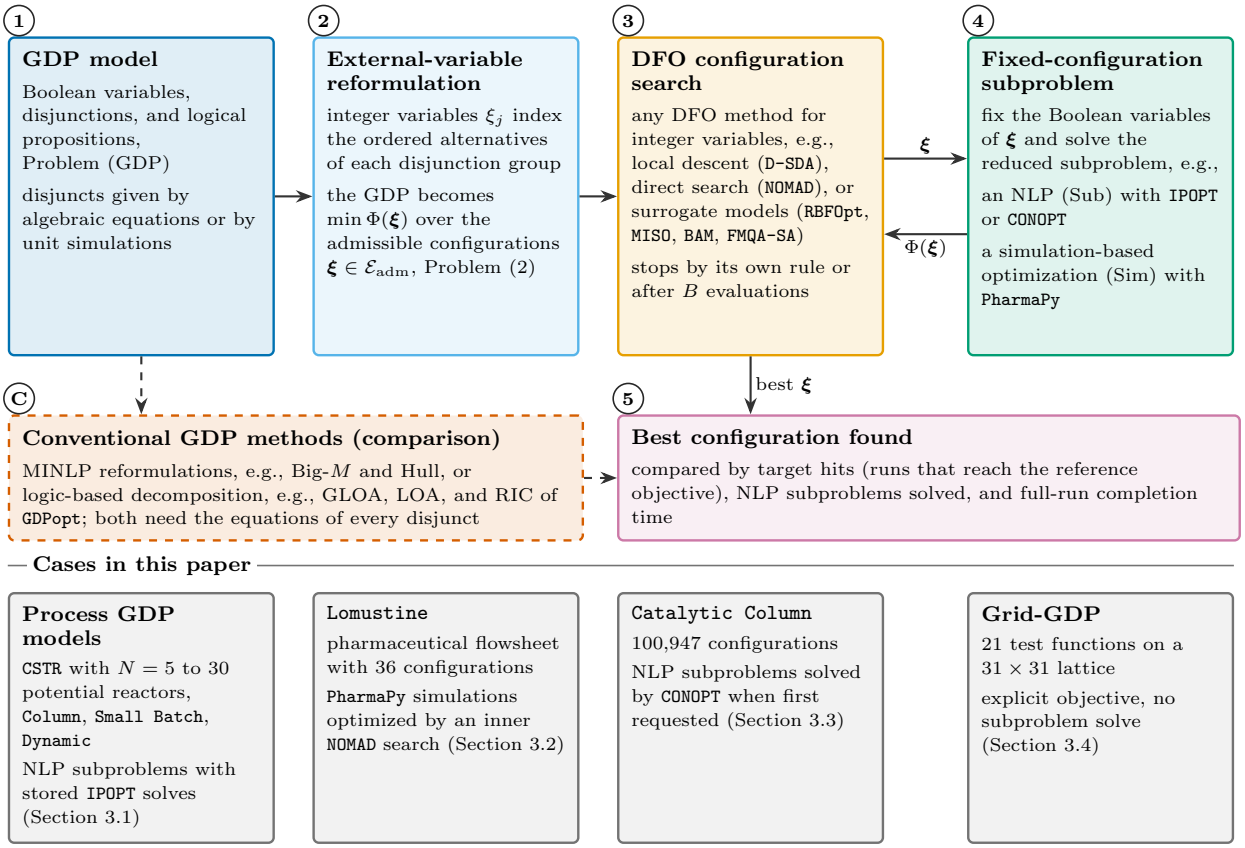


Figure 1: Methodology of this study. Top, the general method: a GDP model (1) is rewritten with external integer variables $\boldsymbol{\xi}$ that index the alternatives of its disjunctions (2), so that solving it becomes a search over the admissible configurations whose objective is the subproblem value $\Phi(\boldsymbol{\xi})$. A DFO method (3) proposes a configuration, which is evaluated by fixing its Boolean variables and solving the reduced subproblem (4), and the returned value updates the search until the method stops or the evaluation budget B is spent; the best configuration found (5) is the result. The dashed path (C) shows the conventional GDP methods, MINLP reformulations or logic-based methods such as global logic-based outer approximation (GLOA), logic-based outer approximation (LOA), and relaxation with integer cuts (RIC), which solve the same models when the equations of every disjunct are available. Bottom: the four cases of this paper and the subproblem evaluation each one uses.

functions, measured by target attainment, NLP subproblems solved, and completion time.

- Evidence from execution records that the logic-based methods RIC and LOA solve one subproblem for each admissible configuration of most of these models, whereas surrogate search reaches near-optimal configurations with a small fraction of those subproblems.
- A benchmark with live NLP evaluations on a catalytic distillation column whose 100,947 configurations are too many to enumerate, together with valid Big-M and Hull reformulations of its GDP model, on which conventional MINLP solvers fail, and global solvers return contradictory certificates.
- An analysis of landscape features that determines when the surrogate search outperforms local descent, with recommendations for choosing between enumeration, direct reformulation, local descent, and surrogate search.

2 Background

2.1 Generalized Disjunctive Programming and Fixed-Configuration Evaluation

GDP represents operational and design alternatives using Boolean variables and associated algebraic constraints (Raman and Grossmann, 1994). The original continuous variables have domain $\mathcal{X} = [\underline{\mathbf{x}}, \overline{\mathbf{x}}] \subseteq \mathbb{R}^{n_x}$. The original integer variables have domain $\mathcal{Z} = [\underline{\mathbf{z}}, \overline{\mathbf{z}}] \cap \mathbb{Z}^{n_z}$. Let K index the disjunctions and D_k index the disjuncts in disjunction $k \in K$. Each disjunct $i \in D_k$ has a Boolean variable Y_{ik}. The GDP formulation is given by

$$\begin{aligned} \min_{\mathbf{x},\mathbf{z},\mathbf{Y}} \quad & f(\mathbf{x},\mathbf{z}) \\ \text{s.t.} \quad & \mathbf{g}(\mathbf{x},\mathbf{z}) \le \mathbf{0}, \\ & \veebar_{i \in D_k} \begin{bmatrix} Y_{ik} \\ \mathbf{h}_{ik}(\mathbf{x},\mathbf{z}) \le \mathbf{0} \end{bmatrix}, \qquad k \in K, \\ & \Omega(\mathbf{Y}) = \text{True}, \\ & \mathbf{x} \in \mathcal{X}, \quad \mathbf{z} \in \mathcal{Z}, \quad \mathbf{Y} \in \{\text{False}, \text{True}\}^{n_y}. \end{aligned} \tag{GDP}$$

Here, f is the objective function, $\mathbf{g}$ represents the global constraints, and $\mathbf{h}_{ik}$ represents the constraints associated with each disjunct. The operator $\veebar$ requires that exactly one disjunct be selected within each disjunction. Logical propositions $\Omega(\mathbf{Y})$ impose additional relationships between Boolean variables.

Following the external-variable reformulation of Ovalle et al. (2025), each ordered group of m_j Boolean alternatives is represented by an external integer variable $\xi_j \in \{1, \ldots, m_j\}$, which identifies the selected alternative. Ordered alternatives, such as the position of a feed along a column, are one of the conditions that Ovalle et al. (2025) use because they make neighboring values of ξ_j physically similar configurations. The representation itself does not require this order: unordered alternatives, such as a choice among equipment types, can be indexed in any fixed order, as in Section 3.2, although the order then shapes the neighborhoods and surrogate models of the search (Section 4.2). Let n_E denote the number of external integer variables, and $\mathcal{Y}_{\text{logic}}$ the set of Boolean assignments that satisfy the conditions `exactly-one`, and $\Omega(\mathbf{Y})$. The external-variable lattice and its logically admissible subset are

$$\begin{aligned} \mathcal{L}_E &= \prod_{j=1}^{n_E} \{1, \ldots, m_j\}, \\ \mathcal{E}_{\text{adm}} &= \{\boldsymbol{\xi} \in \mathcal{L}_E : \mathbf{Y}(\boldsymbol{\xi}) \in \mathcal{Y}_{\text{logic}}\}. \end{aligned} \tag{1}$$

For each logically admissible configuration $\boldsymbol{\xi}$, the corresponding Boolean variables are fixed, yielding the following reduced optimization subproblem:

$$
\begin{aligned}
\Phi(\boldsymbol{\xi}) = \inf_{\mathbf{x},\mathbf{z}} \quad & f(\mathbf{x},\mathbf{z}) \\
\text{s.t.} \quad & \mathbf{g}(\mathbf{x},\mathbf{z}) \le \mathbf{0}, \\
& \mathbf{h}_{ik}(\mathbf{x},\mathbf{z}) \le \mathbf{0} \quad \text{for all } (i,k) \text{ with } Y_{ik}(\boldsymbol{\xi}) = \text{True}, \\
& \mathbf{x} \in \mathcal{X}, \quad \mathbf{z} \in \mathcal{Z}.
\end{aligned} \tag{Sub}
$$

Here, $\Phi(\boldsymbol{\xi})$ represents the optimal objective value of the reduced subproblem. The reduced subproblem is an NLP when all remaining decision variables are continuous. If the reduced subproblem is infeasible, we define $\Phi(\boldsymbol{\xi}) = +\infty$.

When the external-variable mapping represents all logically admissible Boolean configurations, the original GDP can be expressed as the following upper-level optimization problem (Ovalle et al., 2025):

$$
\min_{\boldsymbol{\xi} \in \mathcal{E}_{\text{adm}}} \Phi(\boldsymbol{\xi}). \tag{2}
$$

A logically admissible configuration may still yield an infeasible reduced subproblem. Problem (2) separates the two tasks of solving a GDP: the model defines the admissible configurations and the subproblem of each, and the method that solves Problem (2) decides which configurations to evaluate.

Problem (2) uses only the value $\Phi(\boldsymbol{\xi})$ of each configuration, so it also applies when a configuration is evaluated by a process simulator. Fixing $\boldsymbol{\xi}$ then selects the unit models of a flowsheet, and the simulator $\mathcal{S}_{\boldsymbol{\xi}}$ computes its states $\mathbf{s}$, such as stream compositions and crystal size distributions, from operating inputs $\mathbf{u} \in \mathcal{U}_{\boldsymbol{\xi}}$. The reduced subproblem becomes

$$
\Phi(\boldsymbol{\xi}) = \min_{\mathbf{u} \in \mathcal{U}_{\boldsymbol{\xi}}} \big\{ f(\mathbf{u},\mathbf{s}) : \ \mathbf{s} = \mathcal{S}_{\boldsymbol{\xi}}(\mathbf{u}), \ \mathbf{g}(\mathbf{u},\mathbf{s}) \le \mathbf{0} \big\}, \tag{Sim}
$$

whose objective and constraints are known only through simulation outputs. In `PharmaPy`, for example, each unit is described by differential-algebraic equations (DAEs), and the simulator integrates them unit by unit in the order of the flowsheet. A DAE model can instead be discretized by orthogonal collocation on finite elements into algebraic equations, which turns its subproblem into an NLP; the `Dynamic` benchmark of Section 3.1 is solved this way with `pyomo.dae` (Nicholson et al., 2018). A derivative-free method in $\mathbf{u}$ solves Problem (Sim), so $\Phi(\boldsymbol{\xi})$ is approximated by the best value that this inner search finds within its simulation budget. Logic-based and direct GDP methods require the equations of every disjunct and do not apply to Problem (Sim). The configuration-search methods compared here use it through the same evaluation interface as an NLP subproblem, which Section 3.2 illustrates with a pharmaceutical flowsheet simulated in `PharmaPy` (Casas-Orozco et al., 2021).

2.2 Conventional GDP Solution Approaches

Big-M and Hull reformulations transform GDP models into MINLP formulations by associating Boolean variables Y_{ik} with binary variables y_{ik} and translating logical propositions into algebraic constraints (Grossmann and Lee, 2003; Raman and Grossmann, 1994). In the Big-M reformulation, the disjunctive constraints are expressed as $\mathbf{h}_{ik}(\mathbf{x},\mathbf{z}) \le \mathbf{M}_{ik}(1 - y_{ik})$, where the inequality is applied componentwise. The Big-M values must be sufficiently large to render the corresponding constraints redundant when $y_{ik} = 0$, but excessively large values can weaken the continuous relaxation and cause numerical difficulties.

Hull reformulations introduce disaggregated variables, linking constraints, and bounds scaled by binary indicators (Grossmann and Lee, 2003). Disjunctive constraints are imposed on the disaggregated variables when $y_{ik} = 1$, while $y_{ik} = 0$ forces these variables to zero, making the corresponding disjunct inactive. For convex disjunctive sets, this formulation can provide a stronger continuous relaxation than Big-M, although it introduces additional variables and constraints. For nonconvex GDP problems, the Hull reformulation does not guarantee a convex reformulated model or global optimality (Chen et al., 2022).

Logic-based methods address GDP problems directly by exploiting their logical and disjunctive structure to select discrete configurations and construct the corresponding optimization subproblems. LOA alternates between a mixed-integer master problem and fixed-configuration NLP subproblems (Türkay and Grossmann, 1996). Under suitable convexity and regularity assumptions, the master problem uses linearization cuts to obtain lower bounds, while feasible solutions to the NLP subproblem provide upper bounds. For nonconvex

GDP problems, these linearizations may not provide valid global lower bounds, motivating global logic-based outer approximation (GLOA), which requires globally valid relaxations and subproblem bounds. Bergamini et al. (2005) developed a GLOA approach for process networks using piecewise under- and overestimators.

`GDPopt` provides implementations of logic-based methods within `Pyomo.GDP` (Chen et al., 2022; Pyomo Development Team, n.d.). Its RIC method uses a mixed-integer linear programming (MILP) relaxation to select configurations and adds no-good cuts to exclude evaluated discrete assignments, without generating the successive outer-approximation cuts used by LOA (Pyomo Development Team, n.d.).

`LD-SDA` is an existing alternative that searches ordered external configurations by improving neighborhood moves and discrete line searches (Ovalle et al., 2025). It selects the best improving neighbor, continues along the resulting direction while improvement persists, and resumes neighborhood exploration. The present work uses this discrete-search structure as a reference and compares other configuration-selection strategies and their computational costs.

2.3 Derivative-Free Optimization Solvers

DFO uses function observations without requiring derivatives of the objective with respect to the search variables (Larson et al., 2019). Black-box configuration search selects configurations using their evaluation responses without accessing the subproblem equations. For GDP configuration search, the outer method can be derivative-free, even if its evaluation routine uses a derivative-based nonlinear solver. `D-SDA` also uses this derivative-free interface, although it does not construct a surrogate model. The upper-level search operates over external integer variables that represent the logical decisions in the GDP. Some logical restrictions are embedded in the external-variable reformulation, while additional constraints can be imposed on the configuration space. One approach to DFO uses surrogate models to guide the search. Surrogate methods seek to reduce expensive evaluations by fitting models to observed data, with computational cost arising from both objective evaluations and model-based candidate selection.

Several DFO methods handle integer or categorical variables directly, and Ploskas and Sahinidis (2022) compare many of them on mixed-integer test problems. Mesh adaptive direct search, implemented in `NOMAD`, extends pattern search to mixed variables through user-defined discrete neighborhoods (Abramson et al., 2009; Audet et al., 2022), and `MISO` combines radial basis function surrogates with sampling strategies for integer variables (Müller, 2016). Bayesian optimization has also been adapted to combinatorial spaces, for example by fitting sparse second-order models of binary variables in `BOCS` (Baptista and Poloczek, 2018), which are closely related to the factorization-machine surrogate used here, or by placing kernels on graphs of discrete configurations in `COMBO` (Oh et al., 2019). The direct-search method `NOMAD` polls trial configurations on an adaptive mesh and keeps no global surrogate. We also compare four surrogate-based methods that represent different modeling choices: `RBFOpt` and `MISO` interpolate all observations with global radial basis function models but select candidates differently, `BAM` partitions the domain and fits local algebraic models, and `FMQA-SA` fits a quadratic model over a binary encoding.

2.3.1 `NOMAD`

`NOMAD` implements mesh adaptive direct search (MADS) (Audet et al., 2022). Each poll evaluates trial points on a mesh around the incumbent, and the mesh is refined after unsuccessful polls and coarsened after successful ones; for integer variables, it cannot become finer than the integer spacing (Abramson et al., 2009). By default, `NOMAD` 4 complements the poll with search steps based on local quadratic models, but it does not retain a global surrogate of the objective.

2.3.2 `RBFOpt`

`RBFOpt` is an open-source surrogate optimizer for bounded black-box problems, with support for continuous, integer, and categorical variables (Costa and Nannicini, 2018; Nannicini, 2021). Given N observed objective values in external integer configurations $\boldsymbol{\xi}^{(1)}, \ldots, \boldsymbol{\xi}^{(N)}$, `RBFOpt` constructs a radial basis function (RBF) surrogate model of the form

$$\widehat{\Phi}_N(\boldsymbol{\xi}) = \sum_{r=1}^{N} \lambda_r \varphi\Big(\|\boldsymbol{\xi} - \boldsymbol{\xi}^{(r)}\|_2 \Big) + p(\boldsymbol{\xi}), \tag{3}$$

where $\widehat{\Phi}_N(\boldsymbol{\xi})$ approximates the objective value for a candidate configuration $\boldsymbol{\xi}$, λ_r are interpolation coefficients determined from the observed values, φ is the RBF kernel evaluated at the Euclidean distance between configurations, and $p(\boldsymbol{\xi})$ is a polynomial tail depending on the selected kernel.

Candidate selection balances low predicted values against exploration away from evaluated points. The metric stochastic response surface method (MSRSM) combines scaled surrogate predictions and distances to existing samples, changing their relative weights during the search (Regis and Shoemaker, 2007). New objective evaluations update the model.

2.3.3 `MISO`

`MISO` uses the radial basis function surrogate of Equation (3) but selects candidates with strategies designed for integer and mixed-integer variables (Müller, 2016). Its coordinate perturbation and target value (CPTV) strategy first perturbs randomly chosen coordinates of the best configuration found and scores the candidates by surrogate value and distance to the evaluated points, as MSRSM does. When this stage stops improving, it switches to a target value step, which proposes the configuration at which the surrogate could attain a prescribed target value with the least added bumpiness.

2.3.4 `BAM`

`BAM` combines domain partitioning with local surrogate construction (Ma et al., 2023). The evaluated points define boxes, and a potentially optimal selection procedure identifies regions for further modeling and search. Within selected boxes, Automated Learning of Algebraic Models (`ALAMO`) constructs sparse algebraic models from nearby observations (Cozad et al., 2014). `BARON` optimizes these local models to propose candidates, while box-size exploration maintains broader coverage (Ma et al., 2023; Sahinidis, 1996; The Optimization Firm, 2026). Within `BAM`, `BARON` optimizes these learned models and never receives the GDP itself. Its support for integer restrictions allows for the configuration domains considered here (The Optimization Firm, 2026).

2.3.5 Factorization-Machine-Based Optimization

A second-order factorization machine (FM) models individual feature effects and pairwise interactions through shared latent parameters (Rendle, 2010). For a binary encoding $\mathbf{b} \in \{0,1\}^m$ of an external integer configuration $\boldsymbol{\xi}$, the FM fit surrogate is

$$\widehat{\Phi}_{\mathrm{FM}}(\mathbf{b}) = \beta_0 + \sum_{i=1}^{m} \beta_i b_i + \sum_{i=1}^{m-1} \sum_{j=i+1}^{m} \langle \mathbf{a}_i, \mathbf{a}_j \rangle b_i b_j, \tag{4}$$

where $\widehat{\Phi}_{\mathrm{FM}}(\mathbf{b})$ denotes the predicted training response, m is the number of binary features, β_0 is the intercept, β_i is the linear coefficient of b_i, and $\mathbf{a}_i \in \mathbb{R}^r$ is its latent vector of dimension r. The inner product $\langle \mathbf{a}_i, \mathbf{a}_j \rangle$ determines the pairwise interaction coefficient. The model parameters are learned from the evaluated configurations and their corresponding training responses.

For binary inputs, $b_i^2 = b_i$, the fitted model is a binary quadratic model. Excluding the constant term β_0 gives the following quadratic unconstrained binary optimization (QUBO) problem:

$$\min_{\mathbf{b} \in \{0,1\}^m} \mathbf{b}^{\mathsf{T}} \mathbf{Q} \mathbf{b}, \qquad Q_{ij} = \begin{cases} \beta_i, & i = j, \\ \langle \mathbf{a}_i, \mathbf{a}_j \rangle, & i < j, \\ 0, & i > j. \end{cases} \tag{5}$$

The QUBO models the learned surrogate, and a binary string may decode to a configuration outside the domain or one that violates the logical constraints.

The `FMQA` framework combines FM surrogate fitting with annealing-based candidate generation (Kitai et al., 2020). Based on observed evaluations, the method fits the FM, samples binary candidates from the resulting quadratic model, and evaluates a selected decoded configuration through the objective-evaluation routine before updating the training data.

FMQA-SA uses classical simulated annealing to sample binary candidates from the learned QUBO. Temperature-dependent acceptance of uphill moves supports exploration, while gradual cooling favors lower-energy solutions (Kirkpatrick et al., 1983). In this study, no quantum hardware was used.

3 Experimental Methods

3.1 Process GDP benchmark problems

The process benchmarks include a network of continuous stirred-tank reactors (CSTRs) (Liñán et al., 2020a), a 17-stage distillation column (Ghouse et al., 2018), and a small batch plant (Kocis and Grossmann, 1988). We use their GDP formulations reported by Ovalle et al. (2025), and the nine-stage nonlinear mode-switching model of Peng et al. (2024). Their objectives minimize the total reactor volume (`CSTR`), condenser and reboiler duties, plus the active tray cost (`Column`), the investment cost (`Small Batch`), and the negative integral of the squared state (`Dynamic`). The external variables represent the reactor/feed cutoff and recycle location (`CSTR`), reflux and boilup positions (`Column`), parallel unit counts (`Small Batch`), and final stages of modes 1 and 2 (`Dynamic`). For these process benchmarks, fixing the external choices leaves only continuous variables, so the resulting subproblems are NLPs. The `CSTR` superstructure contains N potential reactor positions, where $N \in \{5, 10, 15, 20, 25, 30\}$. We denote each `CSTR` instance by its size parameter N, such as `CSTR30` for $N = 30$. The DAEs of the `Dynamic` model are discretized with `pyomo.dae` by orthogonal collocation, with 30 finite elements per stage and three Radau collocation points (Nicholson et al., 2018), so each of its fixed-configuration subproblems is an NLP. The reference configurations in external variable coordinates are (N, N) for `CSTR`, $(13, 4)$ for `Column` (physical trays $(14, 5)$), $(2, 2, 1)$ for `Small Batch`, and $(1, 4)$ for `Dynamic`. The full GDP formulations of the `CSTR`, `Column`, and `Small Batch` benchmarks are provided in Appendices A.1-A.3 of Ovalle et al. (2025), respectively, while the `Dynamic` formulation is described in Section IV.C of Peng et al. (2024). External-variable mappings for the three design cases are provided in Appendices B.1, B.2, and B.4 of Ovalle et al. (2025). The `CSTR` volume is reported in liters and the `Small Batch` investment in the source model's dollars. The `Column` uses the source model's weighted cost proxy based on duties and active trays. The model sizes and unit conventions are given in the Supplementary Section S4 (Table S3). `Dynamic` is an abstract benchmark with no calibrated physical units for state or time. Minimizing its negative squared-state integral favors greater integrated state magnitude and tests mode sequencing coupled to dynamic equations.

Table 1 summarizes the original GDP and DFO candidate domains, the number of configurations whose objective values the interior-point NLP solver `IPOPT` (Wächter and Biegler, 2006) returns with an accepted status, the number of configurations with measured evaluation times, and the evaluation budgets. The 980 primary process runs, seven methods with ten seeds in each of the 14 model-budget conditions, evaluate 2,106 distinct configurations over all models and sizes. The process searches use frozen objective tables from fixed-configuration `IPOPT` evaluations; only accepted finite objectives are used as valid lookup objectives. The objective, acceptance status, and point cost are taken from the same recorded evaluation attempt for the same model instance and configuration. Measured coverage includes all configurations in each DFO candidate box, including unsuccessful and structural-rejection evaluations. The evaluation budget B limits the total number of objective evaluations, including initialization, repeated calls, and unsuccessful evaluations. The full GDP box precedes the logical and algebraic feasibility restrictions. `Column` searches $\{8, \ldots, 15\} \times \{1, \ldots, 8\}$ (64 coordinates), the logical domain of its original $\{1, \ldots, 15\}^2$ box. Other cases use their full boxes. The `Column` bounds encode the reflux and boilup placement restrictions; algebraic constraints are still checked during configuration evaluation. All DFO methods use this box, and the `Column` conclusions apply only to it. For `CSTR`, recycle/feed ordering admits $N(N + 1)/2$ logical candidates, but not all yield objective values satisfying the solver acceptance criteria defined in Section 3.6. The `CSTR` comparisons do not isolate scalability: evaluation budgets change with size. `Dynamic` admits only configurations with $\xi_1 < \xi_2$ or $(\xi_1, \xi_2) = (9, 9)$, which leaves 37 logically admissible configurations. `Small Batch` uses all 27 configurations in $\{1, 2, 3\}^3$, whose decisions count parallel mixer, reactor, and centrifuge units. Of these configurations, 15 are rejected during feasibility-based bound tightening (FBBT) preprocessing.

Table 1: Process GDP models: configuration domains, evaluation budgets, reference objectives, and the cost of full enumeration. The full GDP box lists the external-variable values before logical restrictions, and the DFO domain is the part of it that the DFO methods search; it equals the full box except for `Column`, whose DFO domain is its logical domain $\{8,\dots,15\} \times \{1,\dots,8\}$, and every configuration in it has a measured evaluation cost. Accepted configurations are those whose fixed-configuration `IPOPT` solve returns a finite objective, violates no constraint or bound by more than 10^{-5}, and ends with an accepted solver status (Section 3.6), and the reference objective f_{ref} is the lowest accepted objective of the DFO domain. The enumeration time is the summed measured cost of evaluating every configuration of the DFO domain once.

Model	N	Full GDP box	Full box size	DFO domain size	Accepted	B	f_{ref}	Unit	Enumeration time (s)
`CSTR`	5	$\{1,\dots,N\}^2$	25	25	15	8	3.06201	L	30.27
	10		100	100	54	15	2.88954	L	122.06
	15		225	225	120	22	2.82939	L	298.42
	20		400	400	210	29	2.79941	L	553.38
	25		625	625	325	36	2.78148	L	899.39
	30		900	900	464	43	2.76957	L	1,719.00
`Column`	–	$\{1,\dots,15\}^2$	225	64	29	13	19,346.1	none	137.24
`Small Batch`	–	$\{1,2,3\}^3$	27	27	12	10	167,428	$	10.18
`Dynamic`	–	$\{1,\dots,9\}$ $\times\{2,\dots,9\}$	72	72	37	12,15,20, 30,37,50	-212.009	none	47.25

N: number of potential reactors of `CSTR`; B: evaluation budget; f_{ref}: reference objective; –: not applicable. Unit: L, liters of reactor volume; $, dollars of investment; none, no physical unit (the weighted cost of duties and active trays of `Column` and the negative squared-state integral of `Dynamic`).

3.2 Simulation-based benchmark: lomustine manufacturing

The `Lomustine` benchmark selects the flowsheet of a manufacturing process for the anticancer drug lomustine, whose model is a `PharmaPy` simulation (Casas-Orozco et al., 2021, 2023). The superstructure was introduced by Laky and Casas-Orozco (2022), related flowsheet synthesis frameworks are described by Barhate et al. (2025), and Park and Bernal Neira (2026) studied its discrete landscape with Ising-based solvers; we use the corrected-model data of SECQUOIA (2026b). The superstructure contains two reactors in series, each a batch reactor, a CSTR, or a plug-flow reactor (PFR), followed by a batch vaporizer and a crystallizer that is either an unseeded batch cooling crystallizer or a train of one, two, or three mixed-suspension, mixed-product-removal (MSMPR) crystallizers. The external variables ξ_1 and ξ_2 select the two reactors in the order (batch, CSTR, PFR), and ξ_3 selects the crystallizer in the order (batch, one, two, three MSMPR stages), which gives $3 \times 3 \times 4 = 36$ configurations, all logically admissible. These choices are categorical, so this case does not meet the ordering condition of the external-variable reformulation; we index the alternatives in the order of the source enumeration, and the neighborhoods of `D-SDA` and `NOMAD` and the surrogate models act on this order.

Written as a GDP, the flowsheet is

$$
\begin{aligned}
\min_{\mathbf{u},\mathbf{s},\mathbf{Y}} \quad & f(\mathbf{u},\mathbf{s}) \\
\text{s.t.} \quad & \bigvee_{a\in\mathcal{R}} \begin{bmatrix} Y_a^1 \\ \mathbf{s}_1 = \mathcal{S}_a^{\mathrm{R}}(\mathbf{u},\mathbf{s}_0) \end{bmatrix}, \qquad \bigvee_{a\in\mathcal{R}} \begin{bmatrix} Y_a^2 \\ \mathbf{s}_2 = \mathcal{S}_a^{\mathrm{R}}(\mathbf{u},\mathbf{s}_1) \end{bmatrix}, \\
& \mathbf{s}_3 = \mathcal{S}^{\mathrm{V}}(\mathbf{u},\mathbf{s}_2), \qquad \bigvee_{c\in\mathcal{C}} \begin{bmatrix} Y_c^{\mathrm{C}} \\ \mathbf{s}_4 = \mathcal{S}_c^{\mathrm{C}}(\mathbf{u},\mathbf{s}_3) \end{bmatrix}, \\
& \mathbf{g}(\mathbf{u},\mathbf{s}) \le \mathbf{0}, \qquad \mathbf{u}\in\mathcal{U}, \qquad \mathbf{Y}\in\{\text{False},\text{True}\}^{10},
\end{aligned}
\tag{Lom}
$$

where $\mathcal{R} = \{\text{batch}, \text{CSTR}, \text{PFR}\}$ and $\mathcal{C} = \{\text{batch}, \text{one}, \text{two}, \text{three MSMPR stages}\}$. Each disjunct contains the `PharmaPy` model of one unit, which maps the operating inputs $\mathbf{u}$ and the state $\mathbf{s}_{k-1}$ of the incoming stream to the outlet state $\mathbf{s}_k$, and $\mathbf{s}_0$ is the reagent feed. The vaporizer $\mathcal{S}^{\mathrm{V}}$ is present in every configuration, f is the objective defined below, and $\mathbf{g}$ is the solubility path constraint. The external variables ξ_1, ξ_2, and

ξ_3 index the three disjunctions, and fixing them selects one unit model in each, which gives Problem (Sim). The unit models are available only as simulations, so Problem (Lom) is a GDP whose disjuncts have no algebraic form for the conventional GDP methods.

For each configuration, `NOMAD` optimizes 12 to 16 normalized operating inputs, including reactor temperatures and residence times, the reagent concentration, the cooling profile, and the crystallizer residence times, with 60 simulations, which solves Problem (Sim) approximately. The objective $-(\dot{m}/10^{-4} + 2d_{50})$, with the production rate $\dot{m}$ in kg/s and the median crystal size d_{50} in μm, rewards both quantities, and a solubility path constraint prevents crystallization before the crystallizer; all 36 configurations satisfy it. These inner optimizations take 91 to 1,019 s per configuration, so evaluating every configuration takes 5.3 h. All methods read the stored results of these inner optimizations, so they evaluate the same $\Phi(\boldsymbol{\xi})$, and each evaluation is charged its recorded inner-optimization time. The reference configuration is a PFR followed by a batch reactor and the batch crystallizer, $\boldsymbol{\xi} = (3, 1, 1)$, with $f_{\text{ref}} = -487.842$; the second-best configuration replaces the PFR by a CSTR and lies 0.8% above it. The budgets are $B = 10$, 15, and 20, with the shared initialization, seeds, unaccepted-response values, and target rule of the process benchmarks, and `D-SDA` uses the same unit L_∞ neighborhood. Supplementary Section S5 lists the 36 objective values and the data provenance.

3.3 Catalytic distillation column with live NLP evaluations

The `Catalytic Column` benchmark designs a catalytic distillation column that produces ethyl *tert*-butyl ether (ETBE) from ethanol and a mixture of butenes, for which Liñán et al. (2020b) formulated a GDP model and solved it with `D-SDA`. The column has a condenser, 20 internal stages, and a reboiler, and the reflux returns to the top internal stage. The GDP decisions place the ethanol feed at stage f_1, catalyst on three stages $c_1 < c_2 < c_3$, the butenes feed at stage f_2, and the boil-up return at stage b, with $2 \le f_1 \le c_1 < c_2 < c_3 \le f_2 \le b \le 21$. Internal stages below the boil-up return do not exist, so b also sets the number of trays. Each existing stage is an equilibrium stage with tray hydraulics, and each catalytic stage adds the reaction kinetics and a 120 °C limit on the catalyst temperature. Every decision chooses a stage from an ordered list, so this model meets the ordering condition of the external-variable reformulation (Ovalle et al., 2025). We use a corrected version of the published model in which thermodynamic, property, hydraulic, and cost errors are fixed and the pure-component data are fitted to open reference data (Supplementary Section S6). The objective is the negative annual profit, in \$/yr, of a column whose bottoms product contains at least 95 mol% ETBE, at 2010 prices; it is positive because the butenes left in the distillate earn no credit. With the configuration fixed, the model is an NLP with about 2,600 variables and 2,700 equations, which `CONOPT` (Drud, 1994) solves from the initialization of the published model.

The search methods use the six gaps $\mathbf{g} = (f_1{-}2,\ c_1{-}f_1,\ c_2{-}c_1{-}1,\ c_3{-}c_2{-}1,\ f_2{-}c_3,\ b{-}f_2) \in \{0, \dots, 6\}^6$ as external variables, so that every point of the box satisfies the order of the stages. The GDP admits 100,947 configurations, and the bound of six on each gap, which keeps every design of a pilot within 5% of the pilot's best, leaves 54,159 of them in the box. Of the 117,649 points of this box, the 54,159 with $\sum_i g_i \le 17$ are admissible; the others would place the boil-up return below the last internal stage and are charged to the budget without an NLP solve, like the logical rejections of `CSTR` and `Dynamic`. Unlike in the other process cases, no stored table of responses exists: each evaluation solves the NLP of its configuration when the configuration is first requested. Because these responses are deterministic, they are stored and reused when another run requests the same configuration, and each evaluation is charged the recorded solve time of its configuration. A configuration is accepted when `CONOPT` returns an optimal or locally optimal solution with normal completion within 300 s. The 3,499 configurations solved in the campaign took a median of 4.8 s and a mean of 7.9 s, so solving the 54,159 admissible configurations of the box would take about 119 h, and all 100,947 configurations of the GDP about 220 h.

The budgets are $B = 25$, 50, and 100 evaluations, and each seed has seven shared initial configurations drawn uniformly from the admissible configurations, one more than the number of external variables, which replace the three Latin hypercube points of the other benchmarks. `D-SDA` starts from the best of them and uses the separable neighborhood, which changes one external variable by ± 1 and corresponds to the 2-norm neighborhood of the original `D-SDA` study of this column (Liñán et al., 2020b). The unit L_∞ neighborhood of the other benchmarks would contain 728 configurations in six dimensions, more than seven times the largest budget, whereas the separable neighborhood contains at most 12. Because `D-SDA` moves in the gap

coordinates, one move shifts every stage after the changed gap, so its neighbors differ from the neighbors in stage positions. `RBFOpt`, `BAM`, `NOMAD`, and `MISO` keep the settings of the process benchmarks, `FMQA-SA` has a time limit of 7,200 s instead of 1,800 s because the solve times of its evaluations count toward it, and uniform sampling draws without replacement from its own random stream. No global optimum is known, so the reference is the best accepted configuration found by any run or by a pilot of 202 configurations, $(f_1, c_1, c_2, c_3, f_2, b) = (8, 8, 12, 17, 17, 21)$ with $f_{\text{ref}} = 24{,}895.55$ \$/yr, and the target rule is that of the process benchmarks. Its first gap lies on the bound of the box, so we also solved the 747 configurations outside the box within two stages of it: none is better, and the best lies 0.14% above it (Supplementary Section S6). The 210 runs ran as three concurrent single-core jobs on the server described in Section 3.6 with the General Algebraic Modeling System (`GAMS`) (GAMS Development Corporation, 2025) and `CONOPT`.

We also solved three MINLP forms of the column with `DICOPT` (Viswanathan and Grossmann, 1990), `SBB` (GAMS Development Corporation, 2026), `BARON` (Sahinidis, 1996), and `SCIP` (Bolusani et al., 2024) in `GAMS`, each on one core of the same server with a limit of 3,600 s, from the initialization and initial configuration of the published model. These forms contain all 100,947 configurations of the GDP. The first is the MINLP form published with the `D-SDA` study, which multiplies the switched equations by binary variables (Liñán et al., 2020b). The other two are the Big-M and Hull reformulations that the `GAMS` extended mathematical programming framework (Ferris et al., 2009) generates from a disjunctive form of the corrected model with one disjunction for each stage decision. Both reformulations are valid only after two modeling steps. Each nonlinear equation of a disjunct defines a residual variable on every stage, so that every disjunct constraint is linear and the Hull reformulation never evaluates terms such as $1/\rho_L$ at zero. Each disjunct constraint is also divided by an interval bound on its residual, so that $M = 1$ relaxes every constraint whose disjunct is inactive (Supplementary Section S6). These interval bounds are finite only if the liquid is denser than the vapor, so the two reformulations also bound the mass densities of the liquid (at least 200 kg/m^3) and the vapor (5 to 150 kg/m^3), which the GDP evaluations and the published form do not. For any vapor composition, the vapor bounds cannot bind within the model's bounds on pressure, temperature, and compressibility factor, and every validated solution has a liquid density of at least 529 kg/m^3. With the binary variables fixed, the Big-M and Hull reformulations reproduce the GDP objective at the six feasible configurations we tested, the published form reproduces it at four of them and reaches other local optima at the remaining two, and all forms agree that three other configurations are locally infeasible.

3.4 Controlled benchmark: Grid-GDP

To study search behavior without NLP subproblems, we also consider 21 two-dimensional test functions adapted from the Quantum Hamiltonian Descent benchmark collection (Wu Group, n.d.). Each is sampled at 31 equally spaced values per coordinate, giving 961 configurations. For each benchmark function, let $[\ell_x, u_x]$ and $[\ell_y, u_y]$ denote the coordinate bounds, and let $I = J = \{0, \ldots, 30\}$ be the corresponding index sets. The Grid optimizers receive $(i, j) \in \{0, \ldots, 30\}^2$ directly. Relative to the one-based convention in Section 2.1, $(\xi_1, \xi_2) = (i + 1, j + 1)$ is only a relabeling of alternatives. `FMQA-SA` encodes each Grid index in five unsigned bits and rejects decoded values of 31. The physical coordinates are defined as

$$a_i = \ell_x + \frac{i(u_x - \ell_x)}{30}, \qquad b_j = \ell_y + \frac{j(u_y - \ell_y)}{30}. \tag{6}$$

Let f denote the original analytical objective function and let L and U be its minimum and maximum over the 31×31 lattice. Both constants are obtained by exhaustive preprocessing of all 961 lattice points prior to any search. A normalized target tolerance of 10^{-5} corresponds to $10^{-5}(U - L)$ in raw objective units, so equal normalized tolerances do not need to define equally large target sets. Supplementary Table S1 gives target-set sizes for all functions. Full-precision normalization constants are available in the repository

(Supplementary Section S8). Each benchmark function is represented by the following GDP:

$$
\begin{aligned}
\min_{x,y,Y^x,Y^y} \quad & F(x,y) = \frac{f(x,y) - L}{U - L} \\
\text{s.t.} \quad & \bigvee_{i \in I} \begin{bmatrix} Y_i^x \\ x = a_i \end{bmatrix}, \qquad \bigvee_{j \in J} \begin{bmatrix} Y_j^y \\ y = b_j \end{bmatrix}, \\
& \text{ExactlyOne}(Y_0^x, \ldots, Y_{30}^x), \\
& \text{ExactlyOne}(Y_0^y, \ldots, Y_{30}^y), \\
& \ell_x \le x \le u_x, \qquad \ell_y \le y \le u_y, \\
& Y_i^x, Y_j^y \in \{\text{False}, \text{True}\}.
\end{aligned}
\tag{7}
$$

The two exclusive disjunctions contain 62 coordinate-selection Boolean indicators. The objective retains the explicit mathematical expression. The conventional GDP methods therefore receive the expression and its disjunctive structure, whereas the DFO methods observe only the objective values at the coordinates they select.

Fixing both coordinate disjunctions fixes x and y, leaving no remaining original continuous degrees of freedom. The measured fixed-point `IPOPT` (Wächter and Biegler, 2006) evaluations therefore consist of model preparation, solver invocation, and verification. The unfixed objective functions remain nonlinear and generally nonconvex, so Big-M and Hull produce MINLP representations. The complete mathematical definitions and coordinate domains of the 21 benchmark functions are provided in Supplementary Sections S1-S2 (Table S1) (SECQUOIA, 2026a; Wu Group, n.d.).

3.5 Search protocol and compared algorithms

3.5.1 Shared initialization and evaluation budgets

In the process, `Lomustine`, and Grid-GDP benchmarks, all configuration-search methods and sampling controls use precomputed configuration responses during the search; the `Catalytic Column` solves each configuration when it is first requested (Section 3.3). For process cases, accepted responses are objective values from `IPOPT` under the initialization and acceptance rules in Section 3.6. Because `IPOPT` returns local solutions, these values can exceed the subproblem value $\Phi(\boldsymbol{\xi})$.

Each Grid-GDP benchmark uses ten seeds, each defining a different initial design, and evaluation budgets $B \in \{10, 30, 50, 70, 100\}$. The primary Grid comparison comprises six search implementations and two sampling controls, giving $21 \times 10 \times 5 \times 8 = 8{,}400$ runs. The process benchmarks comprise 14 model-budget conditions, each evaluated with ten paired seeds, giving 140 runs per method and 980 runs across the six search implementations and uniform sampling (Table 1). The `Lomustine` benchmark adds three budgets with ten seeds each, 210 runs in total, and the `Catalytic Column` another 210 runs with seven initial configurations per seed (Section 3.3). In all other benchmarks, for each problem-budget-seed combination, three distinct initial configurations are generated using seeded Latin hypercube sampling, mapped to integer coordinates, and shared in the same order across all methods and sampling controls. This common design controls the first three evaluations; subsequent sampling, response handling, and stopping rules follow the individual implementations. All process cases use seeds 0-9, with no replacement of failed starts.

Methods may terminate before exhausting B; only Grid `D-SDA` receives the reference target as a stopping criterion. Detailed settings and worker time limits are provided in the Supplementary Section S3. Every objective evaluation consumes budget, including initialization, repeated configurations, and unaccepted evaluations; internal cache lookups that do not request an evaluation are free. Target attainment uses the recorded acceptance status and original objective values, while each method receives the following values for unaccepted evaluations. In all process cases, `D-SDA`, `RBFOpt`, `NOMAD`, and `MISO` assign a penalty of 10^6 to unaccepted evaluations. `BAM` represents these evaluations by `NaN`, indicating that no accepted objective value is available. For surrogate training, `FMQA-SA` scales the accepted objective values to $[0, 1]$ and assigns the unaccepted evaluations a value of 2.

3.5.2 Budget-limited D-SDA

We use a budget-limited implementation of the previously developed D-SDA (Liñán et al., 2020a), applying its neighborhood search and discrete line search over the GDP external integer lattice as in LD-SDA (Ovalle et al., 2025). It evaluates the unit L_∞ neighborhood in fixed enumeration order, selects the best improving neighbor, and continues along the resulting direction through a discrete line search; on the six-dimensional Catalytic Column, it uses the separable neighborhood instead (Section 3.3), and Supplementary Section S14 repeats every other D-SDA run with the separable neighborhood. Neighborhood and line-search moves require an absolute objective improvement greater than 10^{-5}. As in the original D-SDA and LD-SDA implementations, a configuration is never evaluated twice: when a neighbor or line-search point was already evaluated in the same run, including the shared initial configurations, D-SDA reuses its stored value, and no evaluation is charged. If the evaluation budget B is exhausted, the best solution found is retained.

For the process benchmarks, D-SDA starts with the best accepted configuration among the three initial points evaluated and stops when this descent ends. For Grid-GDP benchmarks, initial configurations are ranked by objective value and successive descents are attempted while evaluation budget remains. After each completed descent, the search stops if the best observed objective satisfies the common target criterion in Equation (8); otherwise, it tries the next ranked initial configuration. For all process benchmarks, D-SDA searches within the declared integer bounds without precomputed feasibility filtering, using the evaluation accounting and unaccepted-response values stated above. To check that the single process descent does not handicap D-SDA, we repeated the 140 process runs with the Grid restart rule, and with further descents from uniformly drawn unvisited configurations until the budget is exhausted (Supplementary Section S13).

3.5.3 NOMAD

NOMAD (Audet et al., 2022), called through PyNomadBBO, runs mesh adaptive direct search with all external variables declared integer. It starts from the best of the three shared initial configurations, whose stored value it reuses without charge, with other settings at their defaults. With integer variables, NOMAD stops when its mesh reaches the integer resolution, so, like D-SDA, it can end before the budget. NOMAD sometimes terminated with a segmentation fault after its last new evaluation. Such a run is repeated with larger evaluation caps, and a rerun is accepted only if it evaluates exactly the same configurations in the same order; 123 of the 1,190 runs completed this way. Twenty-one Grid runs at $B = 100$ crashed on every attempt after identical evaluations; they are scored on the evaluations completed before the crash, and 16 of them had already reached the target (Supplementary Section S12). The Lomustine harness repeats a crashed run once; 12 runs of its option-order and capital-cost repetitions, none in the source order of Table 3, crashed on both attempts after identical evaluations and are scored in the same way (Supplementary Section S5).

3.5.4 RBFOpt

RBFOpt uses MSRSM, automatic RBF selection, and the default genetic candidate-search option (Costa and Nannicini, 2018; Nannicini, 2021). All external variables are integers, and the three shared initial configurations are supplied with the additional package initialization disabled. Search stops at the evaluation budget or the native iteration and stall limits. Other surrogate-minimization operations use standalone Bonmin (Bonami et al., 2008) and IPOPT (Wächter and Biegler, 2006) executables. These calls contribute to search time, but not the objective-evaluation budget.

3.5.5 MISO

MISO uses the CPTV strategy of Müller (2016) as implemented in soogo, with a cubic radial basis function model with a linear polynomial tail, integer coordinates, and a median low-pass filter on large objective values. The three shared configurations and their responses initialize the model. Because soogo does not accept pure-integer domains in its coordinate-perturbation step, our adapter applies the same dynamically dimensioned search perturbation on the integer lattice and disables the continuous local search, which has no continuous coordinates to act on. A linear tail needs $d+1$ affinely independent points, so when the shared initial configurations do not span the space, as with the three initial configurations of the three-dimensional Small Batch and Lomustine cases, uniformly drawn admissible configurations, charged to the budget, are

added until they do. A run stops at the budget, or after ten consecutive iterations without a new evaluation when no unevaluated candidate remains.

3.5.6 BAM

BAM uses local algebraic surrogate models optimized by BARON to generate candidate configurations (Ma et al., 2023; The Optimization Firm, 2026). All external variables are integers. The three shared configurations and their responses are supplied without an additional starting point. The remaining evaluation budget is $B-3$, with other settings at their defaults. On Grid-GDP, the next ten planned evaluations use a Sobol sequence, which distributes points across the domain, subject to the remaining budget. In the process experiments, BAM stops at its own evaluation limit after the B-th evaluation.

3.5.7 FMQA-SA

FMQA-SA uses the FM training and QUBO construction code of SECQUOIA (n.d.). At each iteration, a new second-order rank-eight FM is fitted to all evaluated configurations. Finite precomputed responses are min-max scaled to $[0, 1]$ (or zero when all finite values are identical), while non-finite evaluations receive a training target of 2.0.

Integer coordinates are encoded in unsigned binary, and the fitted FM is converted to a QUBO. Binary candidates are generated on CPU using D-Wave's classical simulated annealing sampler (D-Wave Systems, n.d.). The lowest-energy decoded candidate within the coordinate bounds that has not been previously evaluated is selected; if none is available, a candidate is drawn uniformly from the remaining unevaluated configurations. Candidates are not screened for the feasibility of logical or nonlinear constraints before evaluation. The new observation is appended, and FM fitting and candidate generation repeat until the evaluation budget B or the 1,800 s method time limit is reached.

3.5.8 Sampling controls and full enumeration

Uniform sampling evaluates the same three initial configurations and then draws uniformly without replacement from the remaining declared integer domain. It uses a separate deterministic random stream, no objective-dependent selection, and no logical or NLP-feasibility screening before evaluation. The Grid BAM-prefix control repeats BAM's ten predetermined sampling points in the same order after the three shared starts, then samples uniformly without replacement, stopping at the budget. This control isolates the contribution of the fixed initial design of BAM. Full enumeration evaluates every configuration of the DFO domain once. It reaches the target by construction, because f_{ref} is the best accepted value among these configurations, and its cost is the sum of their measured point costs.

3.6 Execution environment and conventional GDP comparisons

All results reported here come from a server with an AMD EPYC 7643 processor (48 cores, 2.3 GHz) and 1 TB RAM; two earlier runs of Catalytic Column checks on a workstation serve only as comparisons (Supplementary Section S6). All optimization methods and numerical libraries were configured for single-threaded execution. The software comprised Python 3.11.16, Pyomo 6.10.0 (Bynum et al., 2021), RBFOpt 4.3.0, BAM 5.0.0, soogo 2.1.0, NOMAD 4.6.0 called through PyNomadBBO 4.6.0, PyTorch 2.8.0 (Paszke et al., 2019), Bonmin 1.8.9, IPOPT 3.14.19 with MUMPS 5.8.1 (Amestoy et al., 2001), and CONOPT 4.39.1. The process and Grid comparisons called IPOPT and SCIP 9.2.4 (Bolusani et al., 2024) through GAMS 52.2.0. The Catalytic Column used GAMS 54.3.1 with SCIP 10.0.3 and BARON 26.5.27 (Section 3.3).

Fixed-disjunction NLP evaluations. We fix the original Boolean GDP decisions and evaluate the resulting subproblems using IPOPT with the MUMPS linear solver. For the Grid-GDP benchmarks, all 20,181 configurations were evaluated individually, while the process benchmarks used fixed-configuration evaluations of the corresponding process models. IPOPT used a tolerance of 10^{-8} and a solver time limit of 60 s per configuration. Process acceptance requires a finite objective, a maximum absolute violation of any constraint or variable bound not greater than 10^{-5}, and the accepted solver and termination statuses specified in the Supplementary Section S3.1. Time-limited candidates remain unaccepted even when feasible. CSTR sizes

5-30 use the mass-balance/zero-recycle initialization. `Column` and `Small Batch` use their saved initialization files, and `Dynamic` uses fresh clones of the unsolved discretized template.

Conventional GDP comparisons. Conventional comparisons include the `GDPopt` methods GLOA, LOA, and RIC, as well as the Big-M and Hull reformulations. `GDPopt` uses `SCIP` for MILP master problems and `IPOPT` for NLP subproblems (Chen et al., 2022; Pyomo Development Team, n.d.). Because fixing both Grid disjunctions also fixes x and y, the Grid subproblems are solved as linear problems by `SCIP`, and `IPOPT` is not called in those runs. The Big-M and Hull reformulations are solved directly using `SCIP`. All conventional GDP methods use the original GDP domains, a time limit of 1,800 s per instance, and a relative optimality gap of 10^{-5}; the MINLP solvers of the `Catalytic Column` use a limit of 3,600 s and a relative gap of zero (Section 3.3).

Artificial intelligence assistance. `ChatGPT` and `Codex` (OpenAI, including GPT-6 Astra) and `Claude Code` (Anthropic, Claude Opus 5.5) assisted with developing and revising experiment, analysis, and plotting code. The authors reviewed and revised the resulting code and validated the computational analyses. Data visualizations were generated by the documented plotting programs from the archived numerical results. The authors remain responsible for the computational methods and reported findings.

3.7 Target attainment and cumulative success profiles

Let f_{best} denote the lowest feasible objective value observed so far within the evaluation budget, and let f_{ref} denote the reference objective value. For process benchmarks, f_{best} is computed from evaluations that meet the solver acceptance criteria in Section 3.6. A run reaches the reference target when

$$|f_{\text{best}} - f_{\text{ref}}| \leq 10^{-5} \max\{1, |f_{\text{ref}}|\}. \tag{8}$$

For the process experiments, f_{ref} is the lowest accepted objective value among the precomputed fixed-configuration NLP evaluations, so reaching the target means matching the best configuration that full enumeration of the stored responses would find. Runs that terminate with an execution failure are counted as unsuccessful, even if the target was observed earlier; the exception is a `NOMAD` run that crashes on every attempt after evaluating the same configurations, which is scored on the evaluations completed before the crash (Section 3.5.3). For process runs, the reported solution is the best accepted configuration recorded within budget, including the final evaluation. Reported objectives and bounds of the process benchmarks use six significant figures; all target and bound checks use unrounded values. For the full-process-GDP comparison, a feasible objective at or below $f_{\text{ref}} + 10^{-5} \max\{1, |f_{\text{ref}}|\}$ counts as achievement, so solutions better than the reference are also included.

For each evaluation budget B, let $\mathcal{R}_B$ denote the set of problem-seed instances compared across methods, and let s identify a solution method. For each instance $r \in \mathcal{R}_B$, H_{sBr} equals one if the run reaches the target and zero otherwise. Let E_{sBr} be the index of the first objective evaluation at which the target is reached, with $E_{sBr} = +\infty$ if the target is not reached. At a threshold of N objective evaluations, where $0 \leq N \leq B$, the absolute evaluation-count profile is

$$C_{sB}^{\text{eval}}(N) = \frac{1}{|\mathcal{R}_B|} \sum_{r \in \mathcal{R}_B} \mathbf{1}\{H_{sBr} = 1,\ E_{sBr} \leq N\}. \tag{9}$$

Here, $\mathbf{1}\{\cdot\}$ equals one when the stated conditions hold and zero otherwise, and $|\mathcal{R}_B|$ is the number of problem-seed instances. Thus, $C_{sB}^{\text{eval}}(N)$ gives the fraction of runs that reach the target within N evaluations. For the Grid-GDP benchmarks, $|\mathcal{R}_B| = 210$ at each budget; for each process benchmark and budget, $|\mathcal{R}_B| = 10$. All runs, including target misses, remain in the denominator.

Experiments at different evaluation budgets are analyzed separately, so a larger-budget trajectory need not extend a smaller-budget one. These profiles count raw evaluations, without the best-solver normalization of performance profiles (Dolan and Moré, 2002) or the dimension normalization of data profiles (Moré and Wild, 2009).

NLP subproblems solved. An evaluation counts as an NLP subproblem solve when its configuration passes logical and FBBT preprocessing and `IPOPT` is called, whether or not the result is accepted. For `GDPopt`, we count the `IPOPT` calls confirmed in its execution logs. Internal NLP counts are unavailable for the direct Big-M and Hull solves.

3.8 Runtime accounting and uncertainty

For a recorded lookup-based run with n objective evaluations and measured lookup-call times, the estimated full-run completion time is

$$\widehat{T}_{\text{complete}} = T_{\text{recorded}} - \sum_{q=1}^{n} t_q^{\text{lookup}} + \sum_{q=1}^{n} c(\boldsymbol{\xi}^{(q)}) + t_{\text{prep}}. \tag{10}$$

Here, T_{recorded} is the measured runtime of the search based on lookup, t_q^{lookup} is the measured lookup-processing interval of its qth objective call, and $\boldsymbol{\xi}^{(q)}$ is the external integer configuration evaluated in that call. The point cost $c(\boldsymbol{\xi}^{(q)})$ is measured separately for the same benchmark model, problem size and configuration, including configuration-specific preparation, preprocessing, invocation of the solver when required, and validation. The term t_{prep} accounts for the preparation of a single model that is not already included in the point costs. Search time includes surrogate fitting, candidate generation, auxiliary optimization, and recording performed within the method clock. Throughout this manuscript, full-run completion denotes completion of this recorded search stage with reconstructed evaluation costs, excluding final result extraction and export. Every recorded objective call contributes its point cost, including initialization, repeated visits, and preprocessing rejections; the same stored measurement is used each time a configuration is revisited. Completion times therefore combine recorded search paths with one stored cost measurement per configuration.

Grid-GDP timers. Grid-GDP enumeration and objective normalization are excluded from the timing analysis, and $t_{\text{prep}} = 0$ because configuration-specific preparation is included in point costs. Recorded lookup-call times are subtracted from the method runtime for all compared Grid methods. The 21 `NOMAD` runs that crashed on every attempt have no recorded times and are left out of the Grid timing summaries.

Process timers. Structurally rejected configurations incur measured preparation and rejection costs without an NLP solution (Supplementary Section S3.1). For `Dynamic`, the original model-template construction and discretization cost is included once per run through t_{prep}; additional campaign-level preparation is not charged repeatedly. All in-budget evaluations have measured point costs, and estimated full-run completion times are available for all 980 primary process runs.

Conventional GDP completion times are measured separately and are not augmented with the DFO point costs. For Grid-GDP, conventional GDP timings include per-run preparation, reformulation, solution, and validation, but exclude shared-template preparation and lattice enumeration. Full-process-GDP timings include model-template preparation.

Statistical comparisons. With ten seeds per process condition, we compare two methods with the two-sided exact McNemar test on their seed-paired target outcomes, and we call differences that this test does not resolve at the 0.05 level comparable (Supplementary Table S17). The tests of `RBFOpt` against `D-SDA` are descriptive and not adjusted for multiple comparisons, and the 70 exploratory tests that involve `NOMAD` or `MISO` use a Holm correction (Supplementary Section S12). Grid comparisons use a percentile bootstrap that resamples the 21 functions (Supplementary Section S10).

4 Computational Results and Discussion

4.1 Process GDP case studies

4.1.1 Target attainment

Table 2 reports target attainment for the 14 process conditions. Across the 60 paired CSTR runs, RBFOpt reaches the target in 45, MISO in 43, NOMAD in 21, and D-SDA in 16. RBFOpt succeeds in 30 runs in which D-SDA misses, and D-SDA in one in which RBFOpt misses (exact paired test $p < 10^{-7}$; Supplementary Table S17). With ten seeds per size, the difference is significant at the 0.05 level only for CSTR5 and CSTR30, but it has the same sign at every size. On Dynamic at $B = 12$, RBFOpt reaches all ten targets and D-SDA four ($p = 0.031$); D-SDA reaches all ten from $B = 20$ onward.

MISO shows the same advantage over D-SDA with a different profile across sizes. It succeeds in 28 CSTR runs in which D-SDA misses, against one in the other direction ($p < 10^{-6}$), and it reaches all 30 targets on CSTR20 through CSTR30, but only 13 of 30 on CSTR5 through CSTR15, where RBFOpt reaches 21. Pooled over the CSTR sizes, MISO and RBFOpt disagree in 16 runs, seven in favor of MISO ($p = 0.80$). NOMAD reaches more CSTR targets than D-SDA (7 against 2 discordant runs, $p = 0.18$) but fewer than RBFOpt (5 against 29, $p < 10^{-4}$). Like D-SDA, it stops before the budget in many runs (77 of 140) because its mesh reaches the integer resolution. Exploratory per-condition tests of NOMAD and MISO against D-SDA, RBFOpt, and each other find no contrast that remains significant after a Holm correction over the 70 tests (Supplementary Section S12).

On the two smallest design models, D-SDA and RBFOpt are comparable. D-SDA reaches five Column targets and RBFOpt four ($p = 1$), and on Small Batch they reach eight and five ($p = 0.38$). MISO reaches all ten Column targets and NOMAD seven; on Small Batch they reach seven and six. BAM reaches all ten CSTR5 targets, compared with eight for RBFOpt ($p = 0.50$). The only Column hits of BAM and uniform sampling occur in seed 6, whose shared initial design contains the target.

The single-descent D-SDA stops before exhausting its budget in 87 of the 140 process runs. Restarting it from the remaining shared initial configurations, or from uniformly drawn unvisited configurations until the budget is spent, adds one target, raising target attainment from 79 to 80 of 140 runs; both restart variants improve the final best objective in 14 runs (Supplementary Section S13).

FMQA-SA reaches 42 of 140 process targets, compared with 37 for uniform sampling. In the small process domains, the 100 annealed samples often contain no new configuration within the bounds, so 52.9% of the FMQA-SA proposals come from its uniform fallback (Supplementary Section S3.2).

Supplementary Table S12 reports the search times, which exclude configuration-evaluation costs. Median search times stay below 0.06 s for D-SDA and uniform sampling and below 1 s for BAM. For RBFOpt, they grow with the budget, from 0.06 s on CSTR5 to 26.3 s on Dynamic at $B = 50$. MISO needs 0.66 to 158 s because it refits its model and scores a large candidate pool at every iteration, whereas NOMAD stays below 2 s. FMQA-SA needs 3.7 to 29.8 s because it refits its factorization machine after every evaluation.

Supplementary Figure S1 shows the evaluation at which each run that reaches the target first reaches it.

4.1.2 NLP subproblems solved

Figure 2 compares the number of NLP subproblems that each run solves with the number that full enumeration of the configuration domain requires. On CSTR5 through CSTR20 and on Dynamic, RIC and LOA solve exactly as many subproblems as full enumeration, and their execution logs show that every subproblem fixes a different configuration (Supplementary Section S11). RIC also enumerates all 325 admissible CSTR25 configurations and solves 455 of the 465 CSTR30 subproblems before the time limit (Supplementary Table S24). The DFO methods solve at most 38 subproblems in any run, and MISO and RBFOpt reach the CSTR30 target in ten and nine of ten runs with medians of 29.5 and 31.5, 6 to 7% of the enumeration count.

Enumeration does not guarantee that a logic-based method reaches the target. RIC and LOA miss the CSTR10 target after solving all 55 of its subproblems, and RIC misses the CSTR25 target after solving all 325. These misses come from local NLP solutions obtained with the GDPopt initialization, which differs from the initialization used to build the lookup tables (Supplementary Section S3.1). GLOA solves two to four subproblems per CSTR model and misses every CSTR target.

Table 2: Process GDP models: target hits and median full-run completion times of the six DFO methods and uniform sampling, with ten runs per method and condition. A target hit is a run whose best accepted objective lies within $10^{-5}\max\{1, |f_{\text{ref}}|\}$ of the reference objective f_{ref} of Table 1 (Equation (8)). The completion time of a run adds its search time to the measured cost of every evaluated configuration and, for `Dynamic`, one model preparation (Equation (10)); medians are over all ten runs, including target misses. Table 1 gives the enumeration times, and Supplementary Table S12 the search times.

Model	N	B	Measure	D-SDA	NOMAD	RBFOpt	MISO	BAM	FMQA-SA	Uniform
CSTR	5	8	Target hits (of 10)	2	4	8	4	**10**	3	4
			Median completion time (s)	10.67	10.71	10.72	11.33	10.67	13.92	**9.44**
	10	15	Target hits (of 10)	3	3	**6**	2	1	2	1
			Median completion time (s)	**17.61**	21.25	21.79	21.87	18.54	27.07	17.75
	15	22	Target hits (of 10)	2	2	**7**	**7**	1	1	2
			Median completion time (s)	**25.68**	33.92	37.59	39.79	27.67	42.82	27.84
	20	29	Target hits (of 10)	3	2	7	**10**	2	1	2
			Median completion time (s)	**32.31**	38.17	45.03	56.03	39.37	59.10	40.12
	25	36	Target hits (of 10)	3	6	8	**10**	2	1	1
			Median completion time (s)	**45.49**	56.56	62.29	78.56	50.47	74.70	51.17
	30	43	Target hits (of 10)	3	4	9	**10**	2	1	1
			Median completion time (s)	**47.17**	71.23	101.96	107.33	115.41	101.19	65.20
Column	–	13	Target hits (of 10)	5	7	4	**10**	1	4	1
			Median completion time (s)	35.44	31.93	54.60	31.98	27.50	41.82	**21.69**
Small Batch	–	10	Target hits (of 10)	**8**	6	5	7	2	6	4
			Median completion time (s)	4.23	4.97	5.12	5.62	4.23	9.09	**3.49**
Dynamic	–	12	Target hits (of 10)	4	8	**10**	**10**	0	2	0
			Median completion time (s)	12.22	12.36	12.29	11.80	9.95	14.47	**7.80**
		15	Target hits (of 10)	6	**10**	**10**	**10**	3	3	3
			Median completion time (s)	15.32	15.48	16.10	15.44	11.53	18.39	**8.71**
		20	Target hits (of 10)	**10**	**10**	**10**	**10**	3	3	3
			Median completion time (s)	20.02	20.85	20.54	42.42	14.07	25.23	**12.27**
		30	Target hits (of 10)	**10**	**10**	**10**	**10**	4	4	4
			Median completion time (s)	20.02	22.00	34.19	106.24	21.91	36.24	**19.38**
		37	Target hits (of 10)	**10**	**10**	**10**	**10**	7	5	5
			Median completion time (s)	**20.02**	22.00	42.42	111.61	24.99	46.05	24.38
		50	Target hits (of 10)	**10**	**10**	**10**	**10**	**10**	6	6
			Median completion time (s)	**20.02**	21.99	65.14	195.36	33.98	61.31	33.12

N: number of potential reactors of `CSTR`; B: evaluation budget; –: not applicable; Uniform: uniform sampling without replacement. Bold: most target hits and shortest median completion time in each row, including ties.

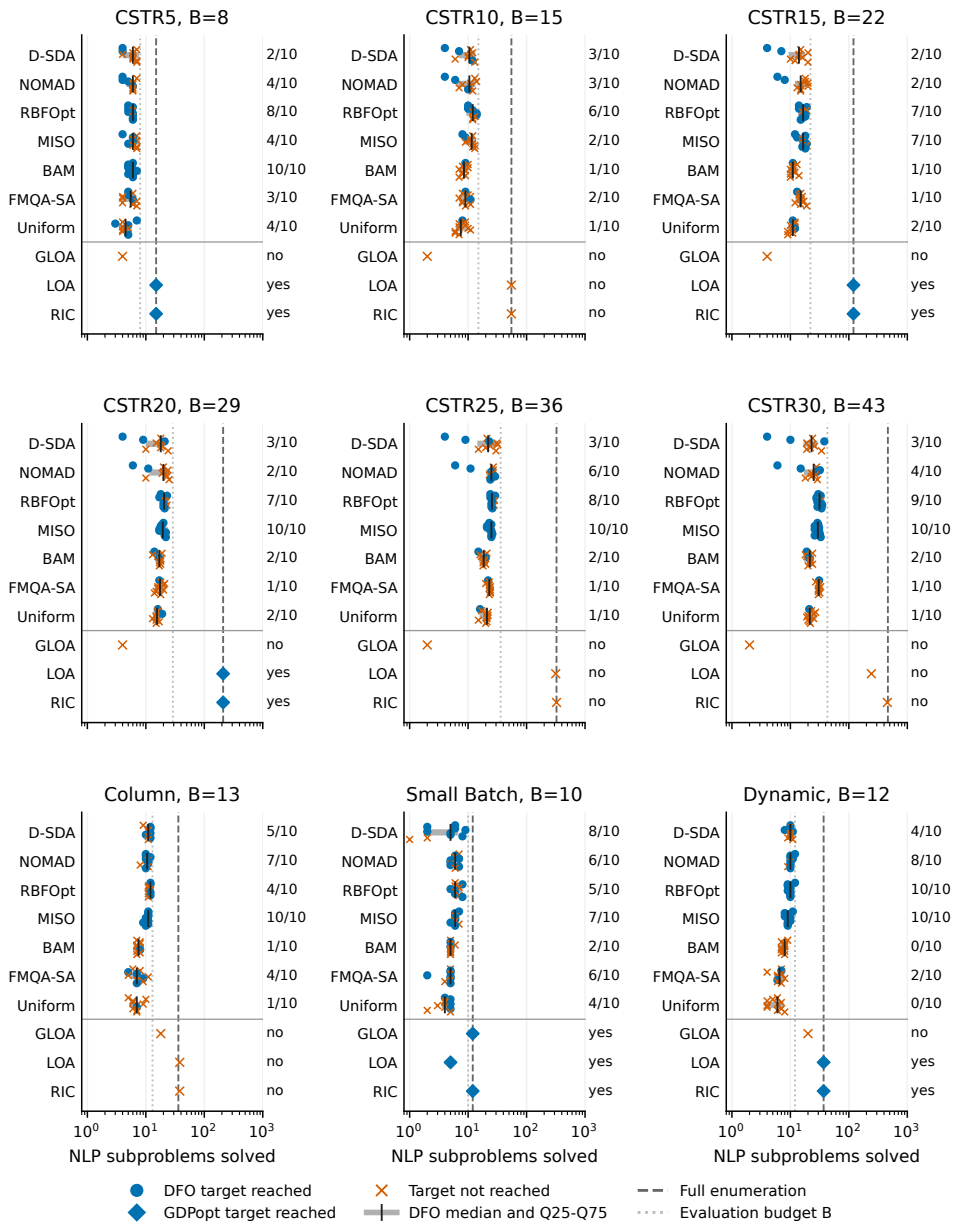


Figure 2: Process GDP models: NLP subproblems solved per run by the six DFO methods and uniform sampling (ten runs each, at the evaluation budget B of each panel title) and by the single run of each `GDPopt` method: GLOA (global logic-based outer approximation), LOA (logic-based outer approximation), and RIC (relaxation with integer cuts). Dots mark DFO runs that reach the reference target of Equation (8), an objective within $10^{-5}\max\{1, |f_{\text{ref}}|\}$ of the reference objective f_{ref} of Table 1; diamonds mark `GDPopt` runs whose feasible objective is at most $f_{\text{ref}} + 10^{-5}\max\{1, |f_{\text{ref}}|\}$ (Section 3.7); crosses mark the other runs; gray bars span the 25th to 75th percentiles of the ten DFO seeds, with a tick at the median. The right-hand labels give the target hits out of ten of each DFO method and the outcome, yes or no, of each `GDPopt` run. The dashed line marks the NLP subproblems needed to evaluate every configuration of the DFO domain, and the dotted line the evaluation budget B. Configurations rejected by logical or feasibility-based bound tightening (FBBT) preprocessing are not counted as NLP subproblems. `GDPopt` searches the full `Column` box of 225 configurations, so its count can exceed the enumeration line of the 64-configuration DFO domain. The direct Big-M and Hull solves are not shown because their internal NLP counts are unavailable. Uniform: uniform sampling without replacement.

4.1.3 Completion time

In wall time, the advantage of fewer subproblem solves depends on how much each solve costs. Configuration evaluations have median costs of 0.4 to 2.5 s in these models (Supplementary Table S19). Full enumeration therefore takes from 10.18 s for `Small Batch` to 1,719.00 s for `CSTR30` (Table 1). Table 2 gives the median completion times of the DFO methods. On `Small Batch`, the DFO medians of 3.49 to 9.09 s are close to the enumeration time, and on `Dynamic` at $B = 50$, `RBFOpt` and `MISO` need 65.14 and 195.36 s, more than enumerating all 72 configurations. On `CSTR30`, `MISO` and `RBFOpt` reach the target in ten and nine of ten runs with medians of 107.33 and 101.96 s, about one sixteenth of the enumeration time, while `NOMAD` and `D-SDA` reach four and three targets with medians of 71.23 and 47.17 s.

Search overhead and stopping rules also shape completion times. For `Dynamic` at $B = 50$, search overhead contributes 42.7% of the mean completion time of `RBFOpt`, compared with less than 0.1% for `D-SDA`, and `MISO` spends a median of 158 s of its 195.36 s in search. Across these ten runs, `D-SDA` and `NOMAD` evaluate 167 and 170 configurations because they stop at a local optimum or at the integer mesh resolution, whereas `RBFOpt`, `MISO`, `BAM`, and `FMQA-SA` each evaluate 500. At $B = 30$, `RBFOpt` reaches all ten `Dynamic` targets in fewer evaluations than `D-SDA` (Supplementary Figure S1), but it continues to the budget, and its median completion time is 34.19 s, compared with 20.02 s for `D-SDA`, 22.00 s for `NOMAD`, and 106.24 s for `MISO`. On `Column`, where `MISO` reaches all ten targets, the median completion times are 31.93 s for `NOMAD`, 31.98 s for `MISO`, 35.44 s for `D-SDA`, and 54.60 s for `RBFOpt`.

Figure 3 places the conventional GDP methods on the same time axis, and Supplementary Table S23 lists their objectives; Supplementary Figures S2 and S3 show the other eight conditions. Hull reaches the target on seven of the nine models, RIC, LOA, and Big-M on five each, and GLOA only on `Small Batch`. Big-M and Hull solve `Small Batch` in about 0.5 s, faster than any DFO run, and they are the only conventional methods that reach the `CSTR30` target, both stopping at the 1,800 s time limit. No conventional method reaches the `Column` target.

All six `CSTR` GLOA runs report `optimal` termination, yet none comes within the relative target tolerance of 10^{-5} (Equation (8)); their objectives exceed the references by 2.2% on `CSTR5` and by 32% on `CSTR30`. Their reported lower bounds exceed independently checked feasible objectives. An earlier configuration that used `SCIP` as a global solver for both master problems and subproblems returned the same six objectives within a relative difference of 3×10^{-5}, so these misses do not come from the local NLP solver. Supplementary Section S9 reports these and the other bound inconsistencies, which affect the interpretation of reported optimality but not the target counts.

4.2 Simulation-based lomustine case

Table 3 reports the `Lomustine` results. `RBFOpt` and `MISO` reach the reference configuration in eight and seven of ten runs with ten evaluations and in all ten runs with 20, and their successful runs find it at a median of the fifth to sixth and the sixth to seventh evaluation. `NOMAD` reaches it in six runs at every budget because, when the budget allows more, it stops after a median of 12.5 evaluations. `D-SDA` reaches it once with ten evaluations: its unit L_∞ neighborhood in three dimensions contains up to 26 configurations, so ten evaluations do not complete the first neighborhood search. With the separable neighborhood, which has at most six neighbors, `D-SDA` reaches the reference in two, five, and five runs at the three budgets (Supplementary Section S14). `BAM` misses every target with ten evaluations, and all ten of its runs end at the same PFR–PFR configuration, 11% above the reference. Of the 96 misses over all methods and budgets, 41 end at this PFR–PFR configuration and 27 at the CSTR–batch configuration, 0.8% above the reference.

In this case, an evaluation costs minutes of simulation, so the completion time is governed by the number of evaluations. Median search times stay below 13 s, whereas the median completion times with ten evaluations range from 1.0 to 1.5 h, compared with 5.3 h for evaluating all 36 configurations. Search adds less than 0.7 s per evaluation in every run, so the methods differ in completion time through the evaluations they choose; `MISO`, for example, completes its ten-evaluation runs in a median of 1.13 h, about as fast as any method.

Because the order of the categorical alternatives is a convention, we repeated the 210 runs with five other orders, including the reversed order and an order by degree of back-mixing (Supplementary Section S5). The order changes the results of several methods. With 20 evaluations, `BAM` reaches the reference in 0 to 10 of the ten runs depending on the order, `D-SDA` in 3 to 8, and `MISO` in 6 to 10, whereas `RBFOpt` reaches it in 9 or

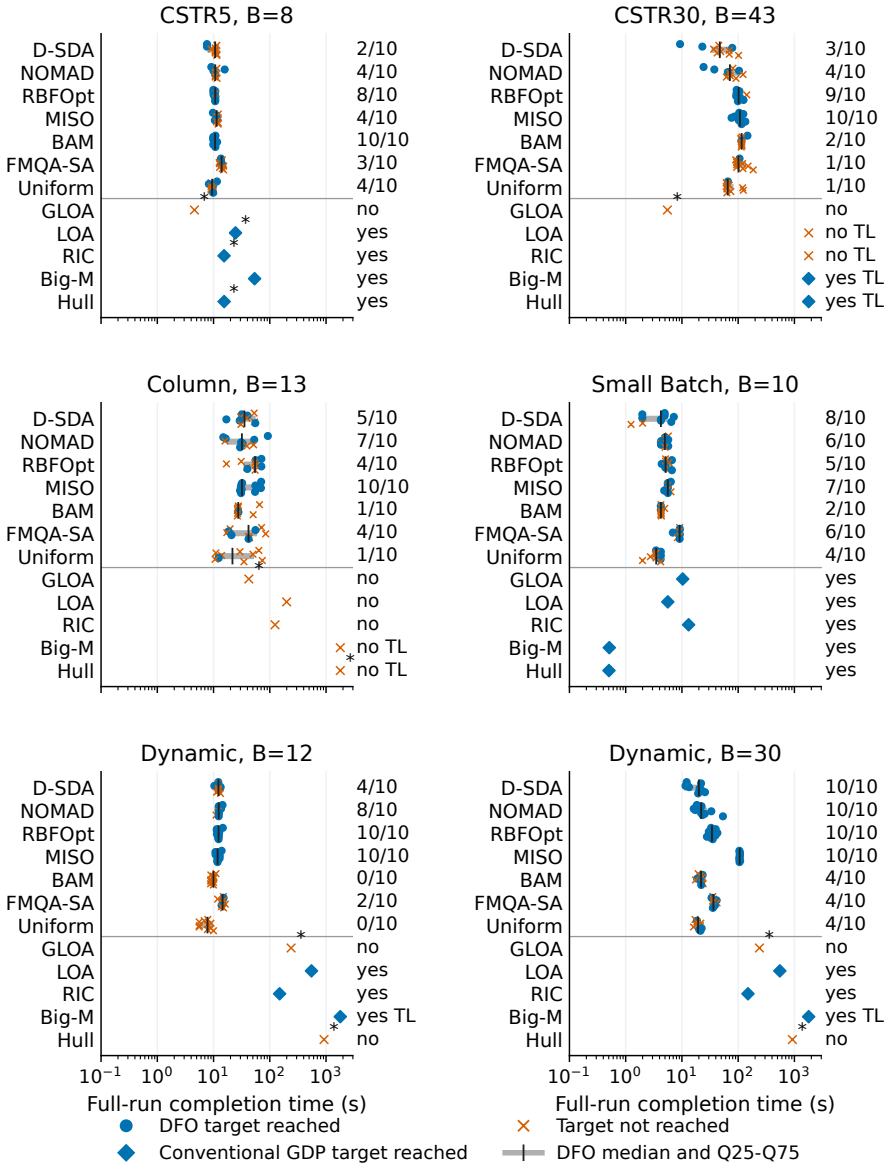


Figure 3: Process GDP models: full-run completion times of the six DFO methods and uniform sampling (ten runs each, at the evaluation budget B of each panel title) and of the single run of each conventional GDP method on six conditions: GLOA (global logic-based outer approximation), LOA (logic-based outer approximation), and RIC (relaxation with integer cuts) of `GDPopt` and the Big-M and Hull reformulations solved by `SCIP`. DFO times add the search time to the measured cost of each evaluated configuration (Equation (10)); conventional GDP times are measured and include model preparation. Dots mark DFO runs that reach the reference target of Equation (8), an objective within $10^{-5}\max\{1, |f_{\text{ref}}|\}$ of the reference objective f_{ref} of Table 1; diamonds mark conventional GDP runs whose feasible objective is at most $f_{\text{ref}} + 10^{-5}\max\{1, |f_{\text{ref}}|\}$ (Section 3.7); crosses mark the other runs; gray bars span the 25th to 75th percentiles of the ten DFO seeds, with a tick at the median. The right-hand labels give the target hits out of ten of each DFO method and the outcome, yes or no, of each conventional run, with TL for runs that report termination at the 1,800 s time limit. Asterisks mark conventional runs whose reported lower bound exceeds a checked feasible objective (Supplementary Section S9). The conventional GDP methods have no evaluation budget, so both `Dynamic` panels show the same conventional runs. Supplementary Figures S2 and S3 show the other eight conditions. Uniform: uniform sampling without replacement.

10 runs in every order. When the objective is the capital cost of each flowsheet at its optimized operation, the all-batch flowsheet is the reference and `BAM` reaches it in every run.

Table 3: `Lomustine`: target hits and median full-run completion times of the six DFO methods and uniform sampling, with ten runs per method and budget. The objective is $-(\dot{m}/10^{-4}+2d_{50})$, and a target hit is a run whose best objective lies within $10^{-5}\max\{1,|f_{\text{ref}}|\}$ of the reference objective $f_{\text{ref}} = -487.842$ (Equation (8)). The reference is the best of the 36 simulated configurations, in which a plug-flow reactor (PFR) is followed by a batch reactor and the batch crystallizer. The completion time of a run adds its search time to the recorded wall time of the `PharmaPy` simulation-based optimization of every evaluated configuration; medians are over all ten runs, including target misses, and evaluating all 36 configurations takes 5.3 h.

B	Measure	`D-SDA`	`NOMAD`	`RBFOpt`	`MISO`	`BAM`	`FMQA-SA`	Uniform
10	Target hits (of 10)	1	6	**8**	7	0	2	2
	Median completion time (h)	1.15	**1.05**	1.24	1.13	1.36	1.47	1.48
15	Target hits (of 10)	4	6	9	**10**	4	3	4
	Median completion time (h)	1.68	**1.22**	1.70	1.82	1.90	2.39	2.11
20	Target hits (of 10)	4	6	**10**	**10**	6	7	5
	Median completion time (h)	1.97	**1.22**	2.44	2.66	2.48	3.09	2.91

$\dot{m}$: production rate (kg/s); d_{50}: median crystal size (μm); f_{ref}: reference objective; B: evaluation budget; Uniform: uniform sampling without replacement. Bold: most target hits and shortest median completion time for each budget, including ties.

4.3 Catalytic distillation column

Table 4 reports the `Catalytic Column` results. No method reaches the reference with 25 evaluations; with 100 evaluations, `D-SDA` and `MISO` reach it in three of ten runs and `RBFOpt` in one. Targets are rare because the 54,159 admissible configurations of the search box are more than 500 times the largest budget and the seven initial configurations are spread over the whole space. The eight best designs accepted in the campaign share the ethanol feed and the first catalytic stage at stage 8, the butenes feed at the last catalytic stage, and the boil-up return at stage 20 or 21, so a run must first reach this region and then refine three stage positions within it.

Most runs that miss the reference with 100 evaluations end close to it. Nine of them, three of `D-SDA`, five of `NOMAD`, and one of `RBFOpt`, end at $(8, 8, 13, 17, 17, 21)$, whose objective is 1.29 \$/yr (0.005%) above the reference. This design differs from the reference only in the second catalytic stage, but moving that stage from 13 to 12 changes two gaps, (g_3, g_4) from $(4, 3)$ to $(3, 4)$. The separable neighborhood of `D-SDA` therefore does not contain the reference, and the descent stops one stage away from it; the unit L_∞ neighborhood contains this diagonal move, but with 728 neighbors it does not fit the budgets. The choice of external variables and neighborhood thus decides which designs are neighbors, as the order of the alternatives did for `Lomustine`: stage positions as coordinates would make this move a single step, but most of their box violates the order of the stages. `MISO` misses by 9.37 to 653 \$/yr, and its median gap of 34.5 \$/yr is close to that of `NOMAD`. `BAM` evaluates, after the shared initial configurations, an initial design of its own that is the same in every seed, and its last point, $(6, 7, 10, 13, 14, 18)$, lies 4.7% above the reference; in six of the ten runs with 100 evaluations, `BAM` finds nothing better.

An evaluation solves an NLP in a median of 4.8 s, whereas the median search overhead per evaluation ranges from 2 ms for uniform sampling to 0.31 s for `MISO` and 0.65 s for `FMQA-SA`, so the completion times are governed by the number of evaluations. With 100 evaluations, the median completion times range from 8.25 min for uniform sampling to 11.4 min for `NOMAD`, and `D-SDA` and `MISO` need 8.71 and 10.5 min. Solving every admissible configuration of the box would take about 119 h, so these runs evaluate 0.2% of them, and the median runs of `D-SDA`, `NOMAD`, and `MISO` end within 0.14% of the reference.

No MINLP solver reaches the reference or comes within 15% of it (Supplementary Table S10). On the published MINLP form, `SBB` stops after 10 s at $(4, 4, 5, 6, 6, 9)$, next to the initial configuration and 15.4% above the reference. `DICOPT` ends without a feasible design on all three forms after 211 to 902 s, and `SBB` finds none on the Big-M and Hull reformulations. `BARON` and `SCIP` declare the published form infeasible after 255 and 9 s, and `BARON` declares the Big-M and Hull reformulations infeasible after 214 and 362 s.

SCIP finds no feasible design on the Big-M and Hull reformulations and stops at the 3,600 s time limit with a lower bound of -5.7×10^7 \$/yr. These infeasibility claims are wrong: with the binary variables fixed at the reference design, BARON still declares both reformulations infeasible, although a CONOPT solution satisfies every constraint and bound of the Big-M reformulation within 10^{-6}. When started from a feasible design, both solvers instead certify that design as the global optimum of the published form, and they do so both for the design of the pilot, at 25,271.33 \$/yr, and for the reference, at 24,895.55 \$/yr, so the first certificate is contradicted by the second design. In every MINLP form, the equations of stages that do not exist and of stages without catalyst stay in the model, multiplied by zero or relaxed, so the solvers must bound terms such as Arrhenius rates over the whole temperature range and inverse powers of vanishing liquid flows; in the Big-M reformulation, the constraint coefficients span 35 orders of magnitude. The GDP evaluation used by the search methods solves only the equations of the selected configuration, and every one of the 210 runs returned accepted designs.

Table 4: Catalytic Column: target hits, gaps to the reference, and median full-run completion times of the six DFO methods and uniform sampling, with ten runs per method and budget. A target hit is a run whose best objective, the negative annual profit, lies within $10^{-5} \max\{1, |f_{\text{ref}}|\}$ of $f_{\text{ref}} = 24{,}895.55$ \$/yr, the best known design, which places the ethanol feed and the first catalytic stage at stage 8, the other catalytic stages at stages 12 and 17, the butenes feed at stage 17, and the boil-up return at stage 21 (Equation (8)). The gap of a run is its best objective minus f_{ref}, and its completion time adds its search time to the recorded CONOPT solve time of every evaluated configuration; both are medians over all ten runs, including target misses. Solving all 54,159 admissible configurations of the search box would take about 119 h.

B	Measure	D-SDA	NOMAD	RBFOpt	MISO	BAM	FMQA-SA	Uniform
25	Target hits (of 10)	0	0	0	0	0	0	0
	Median gap (\$/yr)	1,720	**843**	1,970	1,140	1,170	1,900	1,400
	Median completion time (min)	2.82	2.76	2.38	2.51	2.52	2.94	**2.32**
50	Target hits (of 10)	0	0	**1**	**1**	0	0	0
	Median gap (\$/yr)	1,020	**351**	1,110	946	1,170	1,250	1,190
	Median completion time (min)	5.75	5.35	4.43	5.05	5.13	5.04	**4.00**
100	Target hits (of 10)	**3**	0	1	**3**	0	0	0
	Median gap (\$/yr)	**1.29**	30.4	417	34.5	1,170	1,020	1,120
	Median completion time (min)	8.71	11.4	9.07	10.5	9.52	10.1	**8.25**

B: evaluation budget; Uniform: uniform sampling without replacement. Bold: most target hits when any run reaches the reference, smallest median gap, and shortest median completion time for each budget, including ties.

4.4 Controlled search behavior on Grid-GDP

Grid-GDP isolates search behavior on explicit two-dimensional functions whose configurations need no NLP solve. Table 5 reports the separate experiments at the five budgets. MISO reaches the largest target fraction at $B = 30$, 70, and 100, while RBFOpt leads at $B = 50$ and BAM at $B = 10$. At $B = 100$, MISO and RBFOpt reach 86.2% and 81.4% of the targets, NOMAD 60.0%, and D-SDA 48.1%, with BAM and FMQA-SA in between and uniform sampling at 21.4%. The NOMAD fraction at $B = 100$ includes 16 targets reached before a reproducible solver crash; counting those 21 crashed runs as misses would give 52.4%. The paired bootstrap intervals in Supplementary Section S10 exclude zero for RBFOpt against D-SDA at both $B = 10$ and $B = 100$.

At $B = 10$, BAM and its prefix control each reach 52.9% because nine of the 21 functions have their target at the domain center, which BAM evaluates as the fifth point of its Sobol design. By $B = 100$, the model-based search of BAM adds 23 hits over the prefix control (Figure 4). D-SDA levels off at 48.1% from $B = 70$ onward, once the descents from its three initial configurations end, and its median number of evaluations stays at 37 for every budget from $B = 50$ onward. Excluding Xin-She Yang 3, whose target set covers 870 of the 961 lattice points, and Bohachevsky 2, which nearly duplicates Sum of Squares on the lattice, preserves this ordering.

Because the explicit expression is available, the direct Big-M and Hull reformulations solved with SCIP reach all 21 Grid targets in a median of 0.66 and 0.53 s per function, less than the median completion

Table 5: Grid-GDP: target hits of the six DFO methods, uniform sampling, and the BAM-prefix control at five evaluation budgets, as percentages of 210 runs (21 test functions and ten seeds). A target hit is a run that evaluates a configuration whose normalized objective is within 10^{-5} of zero, the normalized minimum over the lattice (Equation (8)). Each budget is a separate experiment and counts the three shared initial evaluations and repeated configurations. At $B = 10$, all 111 BAM hits precede its surrogate-guided selection, 11 at the shared initial configurations and 100 at two of its predetermined points, and the NOMAD percentage at $B = 100$ includes 16 targets reached before a solver crash that recurred on every attempt.

	Target hits (% of 210 runs)							
B	D-SDA	NOMAD	RBFOpt	MISO	BAM	FMQA-SA	Uniform	BAM-prefix control
10	6.2	18.6	20.0	18.1	**52.9**	7.6	6.7	**52.9**
30	37.1	52.9	61.9	**67.1**	54.8	22.4	11.9	54.8
50	43.3	59.0	**73.3**	70.0	57.6	36.2	13.8	56.7
70	48.1	60.0	76.7	**81.4**	62.9	48.6	17.1	57.1
100	48.1	60.0	81.4	**86.2**	70.0	61.4	21.4	59.0

B: evaluation budget; Uniform: uniform sampling without replacement; BAM-prefix control: the ten predetermined points of BAM after the three shared initial configurations, then uniform sampling (Section 3.5.8). Bold: largest percentage in each row, including ties.

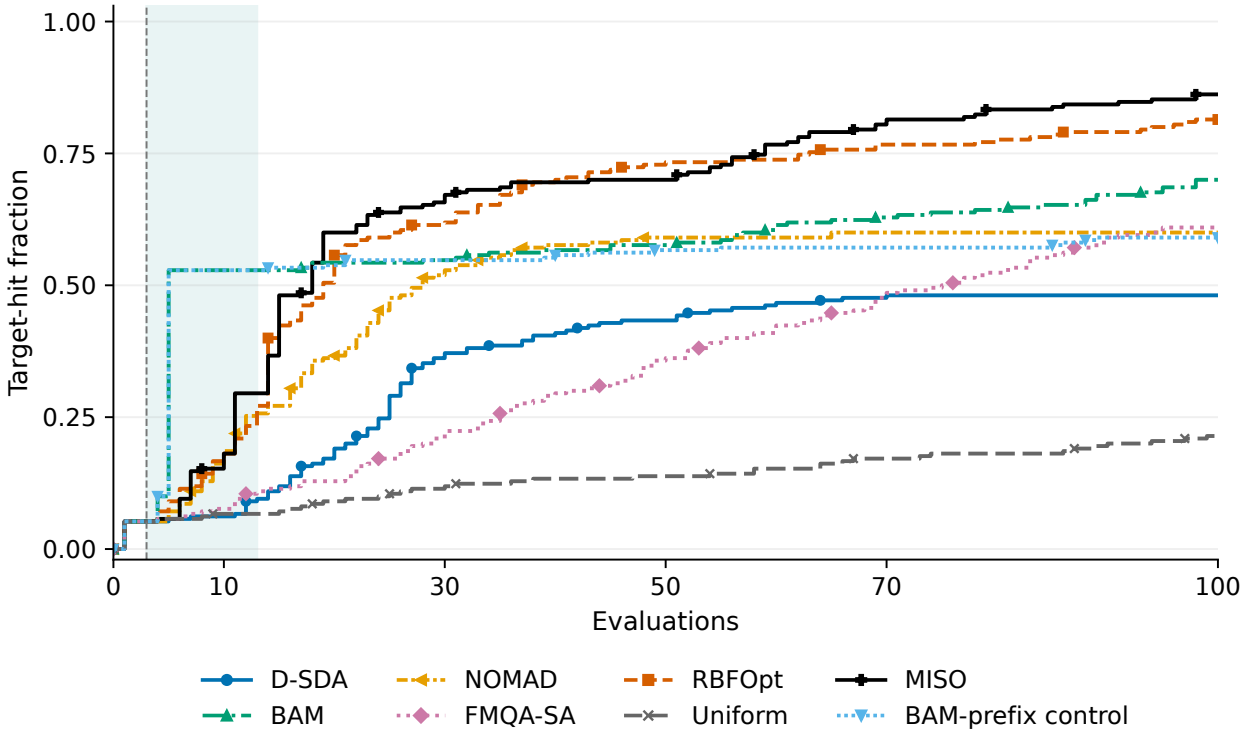


Figure 4: Grid-GDP: fraction of the 210 runs (21 test functions and ten seeds) with the evaluation budget $B = 100$ that have reached the reference target of Equation (8), a configuration whose normalized objective is within 10^{-5} of zero, within a given number of evaluations (Equation (9)). Initial and repeated evaluations count toward the budget, and runs that miss the target remain in the denominator. The runs with $B = 10$, 30, 50, and 70 are separate experiments; at those numbers of evaluations, this profile equals their target-hit percentages in Table 5 for every method except RBFOpt at $B = 10$ (18.1% against 20.0%). The dashed line marks the end of the three shared initial evaluations, and the shaded band the ten predetermined points of BAM, which the BAM-prefix control repeats before it samples uniformly (Section 3.5.8). Uniform: uniform sampling without replacement. At 100 evaluations, MISO reaches the largest fraction, followed by RBFOpt.

time of every DFO method (Supplementary Table S13). The GDPopt methods reach 14 or 15 of the 21 targets, and in diagnostic runs their preprocessing rejected the feasible reference configurations of the three functions that all of them miss. Grid-GDP therefore tests search behavior; for these explicit functions, a direct reformulation is the method of choice.

4.5 Discussion

No single method reaches the most targets in every case. The ranking depends on the size of the configuration space, on where the target lies relative to the initial configurations and to the configurations without accepted objectives, and on how each method uses its evaluation budget.

On CSTR, the recycle/feed ordering makes roughly half of each configuration box logically inadmissible, and the target lies at the corner (N, N), on the edge of the admissible region. On CSTR10 through CSTR30, 31 of the 36 D-SDA misses end at the configuration $(5, 1)$, whose volume of 3.13020 is the same for every N. Because the reference volume decreases with N, this local minimum lies 8.3% above the reference for CSTR10 and 13.0% above it for CSTR30, which explains why the D-SDA gaps grow with size. Restarted descents mostly end at $(5, 1)$ or at the target, so the misses reflect the large basin of this local minimum, and additional budget does not remove them. NOMAD, which polls configurations around its incumbent, ends 32 of its 33 misses on these sizes at $(5, 1)$ as well. RBFOpt and MISO can propose configurations far from their incumbents and reach the corner in nine and ten of the ten CSTR30 runs. A global surrogate does not guarantee success on small budgets, however: MISO reaches 13 of the 30 targets on CSTR5 through CSTR15, with 8 to 22 evaluations, compared with 21 for RBFOpt. On Column, four of the five successful D-SDA runs reach the target in one or two improving moves, and the fifth starts at it. With only 29 and 12 accepted configurations and budgets of 13 and 10 evaluations, Column and Small Batch give a surrogate model few observations to learn from, and local descent remains competitive with RBFOpt. MISO nevertheless reaches all ten Column targets; in the nine runs whose initial design does not contain the target, it finds it between the 7th and 11th of its 13 evaluations. On Grid-GDP, the early lead of BAM reflects the symmetry of the test functions with respect to its initial design, and FMQA-SA improves on uniform sampling there, but in the small process domains, where more than half of its proposals come from its uniform fallback, it reaches only five more targets than uniform sampling (42 against 37 of 140). From $B = 30$ onward, the two radial-basis-function methods, MISO and RBFOpt, reach the most Grid targets.

The methods also use additional evaluations differently. D-SDA terminates when its descents reach configurations without an improving neighbor, and NOMAD when its mesh reaches the integer resolution, so additional budget helps them only until then. On Grid, NOMAD stops after a median of 31.5 evaluations from $B = 50$ onward, and its target fraction stays at 60.0% from $B = 70$ to $B = 100$. The surrogate-based methods and the sampling controls have no comparable termination test and use the full budget in every Grid run, so their Grid target-hit rates keep increasing between $B = 70$ and $B = 100$, by 4.8 percentage points for RBFOpt and MISO, 7.1 for BAM, and 12.9 for FMQA-SA, whereas the D-SDA rate stays at 48.1%. Once a method has found the target, however, the remaining evaluations add cost without improving the result: RBFOpt and MISO reach all ten Dynamic targets at $B = 12$, yet their median completion times grow from 12.29 and 11.80 s at $B = 12$ to 65.14 and 195.36 s at $B = 50$, whereas NOMAD, which stops early, needs 21.99 s at $B = 50$. Additional search time also does not guarantee more targets: on Grid at $B = 100$, FMQA-SA spends a median of 62.04 s on search and reaches fewer targets than BAM, which spends 0.43 s. For MISO, it does: with a median of 32.70 s, about four times the search time of RBFOpt, it reaches 181 targets against 171 (Supplementary Table S13). How much search time is worthwhile depends on the cost of an evaluation. FMQA-SA spends about 0.5 s on search per evaluation, comparable to the evaluations in these models, while RBFOpt spends 17 to 98 ms in median but more as observations accumulate. MISO spends 52 ms to 1.86 s per evaluation, and on Dynamic its search per evaluation exceeds the median evaluation cost of 0.84 s; NOMAD spends 3 to 97 ms (Supplementary Table S19). When each evaluation requires a longer optimization or a process simulation, this overhead becomes small, and the number of subproblem solves becomes the main cost.

Because the initial configurations change with the seed and the surrogate-based methods are randomized, repeated runs can return different solutions, and the spread of these solutions differs between methods. RBFOpt returns at most three distinct final objectives across the ten seeds of each CSTR, Small Batch, and Dynamic condition; on CSTR10 through CSTR30, its runs either reach the target or stop 8.1 to 12.5% above it.

MISO and NOMAD are similarly consistent, with at most five and three distinct final objectives per condition, and the MISO misses on CSTR10 and CSTR15 lie 6.0 to 10.6% above the reference. BAM reaches the exact target in only 18 of the 60 CSTR runs, but all of its final objectives lie within 0.7% of the reference. FMQA-SA and uniform sampling spread more widely, with up to ten distinct final objectives in one condition and gaps of up to 17%. With a relative tolerance of 1%, BAM would succeed in all 60 CSTR runs, compared with 45 for RBFOpt, 43 for MISO, 21 for NOMAD, and 19 for D-SDA (Supplementary Table S18), so the strict target separates exact attainment from near-optimal solutions that may suffice in a design study.

The comparison with enumeration and conventional GDP methods shows how the cost of each approach grows with the configuration space. For small models such as Small Batch and the Grid functions, enumeration or a direct reformulation finds the optimum in seconds or less. As the configuration space grows, logic-based methods that solve one subproblem per admissible configuration and direct reformulations that reach the time limit become expensive, while surrogate search keeps its number of subproblem solves near the budget, as on CSTR30. Dynamic lies between these cases: RIC reaches its target in 149.39 s after solving all 37 admissible subproblems, and RBFOpt and MISO reach it in every run at $B = 12$ in medians of 12.29 and 11.80 s.

The Lomustine case shows that the configuration search carries over when a simulator defines each subproblem and conventional GDP methods do not apply. Its ranking agrees with the CSTR and Grid results: the two radial-basis-function methods reach the reference most often, NOMAD stops early, and the size of the D-SDA neighborhood limits local descent on small budgets. Because each evaluation costs minutes, the search overhead that slows MISO on Dynamic is negligible here. Unordered alternatives can still be encoded as external variables, but their order is then part of the problem the search sees: with 20 evaluations, only RBFOpt reached the reference in at least nine of ten runs in every order.

The neighborhood of D-SDA trades the number of evaluations per step against the moves it can make. Repeating every stored-response D-SDA run with the separable neighborhood, which changes one external variable at a time, shows no overall winner (Supplementary Section S14). The separable neighborhood reaches more targets when the budget barely covers the L_∞ neighborhood, as on Grid at $B = 10$ (22 against 13 of 210 runs) and on Lomustine (99 against 90 of 180 runs over the six orders). It reaches fewer when a descent must move diagonally: on CSTR, 7 against 16 of 60 runs, because 18 of its misses stop on the edge of the admissible region that leads to the target. The number of L_∞ neighbors grows as $3^d - 1$ with the number d of external variables, which is why the Catalytic Column, with six external variables, uses the separable neighborhood.

The Catalytic Column tests the search where enumeration is impractical and every evaluation is a live NLP solve. With 100 evaluations, 0.2% of the configurations in the search box, the median runs of D-SDA, MISO, and NOMAD end within 0.14% of the best known design, and local descent is competitive with the surrogate methods here because the good designs form one region: six of the ten D-SDA descents end at the reference or one stage from it. Within that region, the external variables and the neighborhood decide which designs are neighbors: in the gap coordinates, the runner-up and the reference are two coordinates apart although they differ in one stage, so the separable neighborhood stops local descent 1.29 \$/yr short of the reference, as the order of categorical alternatives changed the results of Lomustine. The MINLP forms of the same model keep the equations of every possible stage and catalyst position, and the solvers fail in different ways: SBB stops at its limit on function evaluations outside their domains, DICOPT when its NLP subproblems fail or are all infeasible, SCIP at the time limit on the Big-M and Hull forms, and BARON and SCIP with infeasibility claims or certificates that the feasible designs contradict. A plausible common cause is that terms harmless in a solved configuration, such as reaction rates on stages without catalyst and hydraulics with vanishing flows, make the relaxations poorly scaled, with constraint coefficients that span 35 orders of magnitude in the Big-M form; the size of the full MINLP, which contains all 100,947 configurations, may contribute as well. Reported bounds were also unreliable in the process models, where GLOA terminated as optimal on every CSTR network while missing the reference by 2.2 to 32%. The GDP model avoids these terms because each evaluation contains only the equations of the selected configuration, so the external-variable representation lets a black-box method optimize a model that its direct MINLP reformulations cannot solve reliably.

Except for the Catalytic Column, these conclusions rest on configuration spaces of 25 to 961 configurations, on NLP responses obtained with one initialization per model, and on completion times that combine recorded search paths with one cost measurement per configuration. The Catalytic Column reference is

the best design found and has no reliable optimality certificate, because the global solvers that certify it also certify a worse design, and its responses also come from one initialization. Because the `CSTR` budgets grow with N, the `CSTR` series does not isolate the effect of size. With ten seeds per condition, the per-condition comparisons that involve `NOMAD` or `MISO` are not significant after a Holm correction over the 70 exploratory tests, and the per-condition tests of `RBFOpt` against `D-SDA` are not adjusted for multiple comparisons, so rankings within a single condition are descriptive. The `NOMAD` and `MISO` search times were measured in a later campaign on a shared server, so small timing differences between them and the other methods should not be overinterpreted.

5 Conclusions

This study modeled process design alternatives as GDP models and searched their configurations with derivative-free methods, using the external-variable reformulation as the interface between the two, and compared this pairing with local descent, conventional GDP methods, and full enumeration. Five lessons follow from the experiments.

First, separating the model from the search extends GDP to models that conventional GDP methods cannot handle. A configuration search needs only the value of each configuration, so the same GDP model can be solved when its subproblems are simulations, as for the `Lomustine` flowsheet, or when its MINLP reformulations fail, as for the `Catalytic Column`, whose evaluations contain only the equations of the selected design.

Second, surrogate search spends subproblem solves economically. RIC and LOA solved one subproblem for each admissible configuration on most process models, whereas the two radial-basis-function methods reached the best configuration of the largest reactor network with a small fraction of those solves.

Third, the landscape and its representation decide whether a global search pays off. Local descent can be competitive when the good configurations form one region, as it was in reaching designs close to the reference of the `Catalytic Column`, and surrogate methods that interpolate all observations are the safer choice when large basins of local minima lie between the initial configurations and the optimum. The external variables, the order of categorical alternatives, and the neighborhood determine which configurations are close, so they shape the landscape that every method sees.

Fourth, exact target attainment is an incomplete measure for stochastic methods. A method that rarely matches the reference can still end consistently close to it, so the spread of final objectives across runs belongs next to the target counts when methods are compared for design studies.

Fifth, the value of search effort depends on the cost of an evaluation. Additional search effort per evaluation can improve target attainment, but its overhead can dominate the run time when evaluations are cheap, and methods that stop at a local criterion leave part of the budget unused; as evaluations become more expensive, the number of subproblem solves becomes the cost that matters.

For future GDP instances, these results suggest enumerating the configurations or solving a direct Big-M or Hull reformulation when the model is small, as for Grid-GDP and `Small Batch`. When the configuration space is too large to enumerate and contains large basins of local minima, as for the larger `CSTR` networks, surrogate-based configuration search with radial basis functions is preferable; in these experiments, `MISO` reached slightly more targets when its search overhead was small relative to the evaluation cost, and `RBFOpt` was the less sensitive to the order of categorical alternatives. For local descent, the unit L_∞ neighborhood suits two external variables and budgets that cover its neighbors, and the separable neighborhood suits more external variables or smaller budgets. When the subproblems are simulations, or when the MINLP reformulations keep equations that are ill-defined for absent units, the GDP model is best solved by configuration search, in which local descent can be competitive when the good designs form one region and surrogate search is the safer choice when the landscape has several basins. Because the wall-time advantage of searching fewer configurations grows with the cost of each evaluation, testing these recommendations on live simulation-based subproblems, in which every evaluation runs the inner optimization, is the next step.

CRediT authorship contribution statement

Albert Joon Lee: Software, Investigation, Formal analysis, Validation, Visualization, Writing – original draft, Writing – review and editing. **Woosik Kim**: Methodology, Software (factorization-machine surrogate optimization with simulated annealing), Validation, Visualization. **David E. Bernal Neira**: Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Resources, Data curation, Visualization, Supervision, Project administration, Funding acquisition, Writing – original draft, Writing – review and editing.

Declaration of competing interests

The authors declare that they have no competing financial interests or personal relationships that could have influenced the work reported in this paper.

Funding

This work was supported by the Davidson School of Chemical Engineering at Purdue University. The funder had no role in study design, data collection, analysis or interpretation, preparation of the manuscript, or the decision to submit it for publication.

Acknowledgements

The authors thank the Davidson School of Chemical Engineering at Purdue University for providing a supportive research environment.

Data and code availability

Sanitized code and data supporting this study are available to reviewers on request from the corresponding author. Public release is planned through the Black-Box-for-GDP repository (SECQUOIA, 2026a). Supplementary Section S8 identifies the archived inputs, including the CSTR updates, Column logical-domain runs, conventional GDP comparisons, `D-SDA` reruns, the `Lomustine` runs, and the `Catalytic Column` runs and MINLP comparisons.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

The authors used `ChatGPT` and `Codex` (OpenAI), including the GPT-6 Astra model, to assist with drafting, organizing, and revising the manuscript and developing analysis and plotting code. The authors also used `Claude Code` (Anthropic), with the Claude Opus 5.5 model, to review the manuscript, revise the Discussion and Conclusions, and develop analysis, plotting, and experiment code. The resulting text and code were checked and revised by the authors, who remain responsible for the article's content.

References

Abramson, M.A., Audet, C., Chrissis, J.W., Walston, J.G., 2009. Mesh adaptive direct search algorithms for mixed variable optimization. Optim. Lett. 3, 35–47. doi:10.1007/s11590-008-0089-2.

Amestoy, P.R., Duff, I.S., Koster, J., L'Excellent, J.Y., 2001. A fully asynchronous multifrontal solver using distributed dynamic scheduling. SIAM Journal on Matrix Analysis and Applications 23, 15–41. doi:10.1137/S0895479899358194.

Audet, C., Le Digabel, S., Rochon Montplaisir, V., Tribes, C., 2022. Algorithm 1027: NOMAD version 4: Nonlinear optimization with the MADS algorithm. ACM Trans. Math. Softw. 48, 35:1–35:22. doi:10.1145/3544489.

Baptista, R., Poloczek, M., 2018. Bayesian optimization of combinatorial structures, in: Proceedings of the 35th International Conference on Machine Learning, pp. 462–471.

Barhate, Y., Laky, D.J., Casas-Orozco, D., Reklaitis, G.V., Nagy, Z.K., 2025. Hybrid rule-based and optimization-driven framework for the synthesis of end-to-end optimal pharmaceutical processes. AIChE Journal , e18888doi:10.1002/aic.18888.

Bergamini, M.L., Aguirre, P., Grossmann, I., 2005. Logic-based outer approximation for globally optimal synthesis of process networks. Comput. Chem. Eng. 29, 1914–1933. doi:10.1016/j.compchemeng.2005.04.003.

Bolusani, S., Besançon, M., Bestuzheva, K., Chmiela, A., Dionísio, J., Donkiewicz, T., van Doornmalen, J., Eifler, L., Ghannam, M., Gleixner, A., Graczyk, C., Halbig, K., Hedtke, I., Hoen, A., Hojny, C., van der Hulst, R., Kamp, D., Koch, T., Kofler, K., Lentz, J., Manns, J., Mexi, G., Mühmer, E., Pfetsch, M.E., Schlösser, F., Serrano, F., Shinano, Y., Turner, M., Vigerske, S., Weninger, D., Xu, L., 2024. The SCIP Optimization Suite 9.0. ZIB Report 24-02-29. Zuse Institute Berlin. URL: https://nbn-resolving.org/urn:nbn:de:0297-zib-95528.

Bonami, P., Biegler, L.T., Conn, A.R., Cornuéjols, G., Grossmann, I.E., Laird, C.D., Lee, J., Lodi, A., Margot, F., Sawaya, N., Wächter, A., 2008. An algorithmic framework for convex mixed integer nonlinear programs. Discrete Optimization 5, 186–204. doi:10.1016/j.disopt.2006.10.011.

Bynum, M.L., Hackebeil, G.A., Hart, W.E., Laird, C.D., Nicholson, B.L., Siirola, J.D., Watson, J.P., Woodruff, D.L., 2021. Pyomo: Optimization Modeling in Python. volume 67 of *Springer Optimization and Its Applications*. 3 ed., Springer. doi:10.1007/978-3-030-68928-5.

Casas-Orozco, D., Laky, D., Wang, V., Abdi, M., Feng, X., Wood, E., Laird, C., Reklaitis, G.V., Nagy, Z.K., 2021. PharmaPy: An object-oriented tool for the development of hybrid pharmaceutical flowsheets. Computers & Chemical Engineering 153, 107408. doi:10.1016/j.compchemeng.2021.107408.

Casas-Orozco, D., Laky, D.J., Wang, V., Abdi, M., Feng, X., Wood, E., Reklaitis, G.V., Nagy, Z.K., 2023. Techno-economic analysis of dynamic, end-to-end optimal pharmaceutical campaign manufacturing using PharmaPy. AIChE Journal 69, e18142. doi:10.1002/aic.18142.

Chen, Q., Johnson, E.S., Bernal, D.E., Valentin, R., Kale, S., Bates, J., Siirola, J.D., Grossmann, I.E., 2022. Pyomo.GDP: an ecosystem for logic based modeling and optimization development. Optim. Eng. 23, 607–642. doi:10.1007/s11081-021-09601-7.

Costa, A., Nannicini, G., 2018. RBFOpt: an open-source library for black-box optimization with costly function evaluations. Math. Program. Comput. 10, 597–629. doi:10.1007/s12532-018-0144-7.

Cozad, A., Sahinidis, N.V., Miller, D.C., 2014. Learning surrogate models for simulation-based optimization. AIChE J. 60, 2211–2227. doi:10.1002/aic.14418.

D-Wave Systems, n.d. dwave-samplers: Classical algorithms for solving binary quadratic models. URL: https://github.com/dwavesystems/dwave-samplers/releases/tag/1.6.0. Version 1.6.0; classical CPU simulated annealing; accessed October 15, 2026.

Dolan, E.D., Moré, J.J., 2002. Benchmarking optimization software with performance profiles. Math. Program. 91, 201–213. doi:10.1007/s101070100263.

Drud, A.S., 1994. CONOPT—a large-scale GRG code. ORSA Journal on Computing 6, 207–216. doi:10.1287/ijoc.6.2.207.

Ferris, M.C., Dirkse, S.P., Jagla, J.H., Meeraus, A., 2009. An extended mathematical programming framework. Computers & Chemical Engineering 33, 1973–1982. doi:10.1016/j.compchemeng.2009.06.013.

GAMS Development Corporation, 2025. General Algebraic Modeling System (GAMS). URL: https://www.gams.com/. release 52.2.0, Fairfax, VA, USA.

GAMS Development Corporation, 2026. SBB solver manual. URL: https://www.gams.com/latest/docs/S_SBB.html. gAMS release 54.

Ghouse, J.H., Chen, Q., Zamarripa, M.A., Lee, A., Burgard, A.P., Grossmann, I.E., Miller, D.C., 2018. A comparative study between gdp and nlp formulations for conceptual design of distillation columns, in: Computer Aided Chemical Engineering, pp. 865–870. doi:10.1016/B978-0-444-64241-7.50139-7.

Grossmann, I.E., Lee, S., 2003. Generalized convex disjunctive programming: Nonlinear convex hull relaxation. Comput. Optim. Appl. 26, 83–100. doi:10.1023/A:1025154322278.

Kirkpatrick, S., Gelatt, Jr., C.D., Vecchi, M.P., 1983. Optimization by simulated annealing. Science 220, 671–680. doi:10.1126/science.220.4598.671.

Kitai, K., Guo, J., Ju, S., Tanaka, S., Tsuda, K., Shiomi, J., Tamura, R., 2020. Designing metamaterials with quantum annealing and factorization machines. Phys. Rev. Res. 2, 013319. doi:10.1103/PhysRevResearch.2.013319.

Kocis, G.R., Grossmann, I.E., 1988. Global optimization of nonconvex mixed-integer nonlinear programming (minlp) problems in process synthesis. Industrial & Engineering Chemistry Research 27, 1407–1421. doi:10.1021/ie00080a013.

Laky, D., Casas-Orozco, D., 2022. Superstructure synthesis and simulation-optimization of pharmaceutical manufacturing processes using PharmaPy. AIChE Annual Meeting, Phoenix, AZ, paper 183b.

Larson, J., Menickelly, M., Wild, S.M., 2019. Derivative-free optimization methods. Acta Numerica 28, 287–404. doi:10.1017/S0962492919000060.

Liñán, D.A., Bernal, D.E., Ricardez-Sandoval, L.A., Gómez, J.M., 2020a. Optimal design of superstructures for placing units and streams with multiple and ordered available locations. part i: A new mathematical framework. Computers & Chemical Engineering 137, 106794. doi:10.1016/j.compchemeng.2020.106794.

Liñán, D.A., Bernal, D.E., Ricardez-Sandoval, L.A., Gómez, J.M., 2020b. Optimal design of superstructures for placing units and streams with multiple and ordered available locations. part ii: Rigorous design of catalytic distillation columns. Computers & Chemical Engineering 139, 106845. doi:10.1016/j.compchemeng.2020.106845.

Ma, K., Rios, L.M., Bhosekar, A., Sahinidis, N.V., Rajagopalan, S., 2023. Branch-and-Model: a derivative-free global optimization algorithm. Comput. Optim. Appl. 85, 337–367. doi:10.1007/s10589-023-00466-3.

Moré, J.J., Wild, S.M., 2009. Benchmarking derivative-free optimization algorithms. SIAM J. Optim. 20, 172–191. doi:10.1137/080724083.

Müller, J., 2016. MISO: mixed-integer surrogate optimization framework. Optim. Eng. 17, 177–203. doi:10.1007/s11081-015-9281-2.

Nannicini, G., 2021. On the implementation of a global optimization method for mixed-variable problems. Open J. Math. Optim. 2, 1–25. doi:10.5802/ojmo.3.

Nicholson, B., Siirola, J.D., Watson, J.P., Zavala, V.M., Biegler, L.T., 2018. pyomo.dae: a modeling and automatic discretization framework for optimization with differential and algebraic equations. Mathematical Programming Computation 10, 187–223. doi:10.1007/s12532-017-0127-0.

Oh, C., Tomczak, J.M., Gavves, E., Welling, M., 2019. Combinatorial Bayesian optimization using the graph Cartesian product, in: Advances in Neural Information Processing Systems.

Ovalle, D., Liñán, D.A., Lee, A., Gómez, J.M., Ricardez-Sandoval, L., Grossmann, I.E., Bernal Neira, D.E., 2025. Logic-Based Discrete-Steepest Descent: A solution method for process synthesis Generalized Disjunctive Programs. Comput. Chem. Eng. 195, 108993. doi:10.1016/j.compchemeng.2024.108993.

Park, Y., Bernal Neira, D.E., 2026. A computational study of Ising-based solvers for discrete landscape exploration in process optimization. AIChE Journal , e70656doi:10.1002/aic.70656.

Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison, A., Kopf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., Bai, J., Chintala, S., 2019. PyTorch: An imperative style, high-performance deep learning library, in: Advances in Neural Information Processing Systems, pp. 8024–8035.

Peng, Z., Lee, A., Bernal Neira, D.E., 2024. Addressing discrete dynamic optimization via a logic-based discrete-steepest descent algorithm, in: 2024 IEEE 63rd Conference on Decision and Control (CDC), IEEE. pp. 1664–1669. doi:10.1109/CDC56724.2024.10886477.

Ploskas, N., Sahinidis, N.V., 2022. Review and comparison of algorithms and software for mixed-integer derivative-free optimization. J. Glob. Optim. 82, 433–462. doi:10.1007/s10898-021-01085-0.

Pyomo Development Team, n.d. GDPopt logic-based solver. URL: https://pyomo.readthedocs.io/en/6.10.0/explanation/solvers/gdpopt.html. Version 6.10.0; accessed October 20, 2026.

Raman, R., Grossmann, I.E., 1994. Modelling and computational techniques for logic based integer programming. Comput. Chem. Eng. 18, 563–578. doi:10.1016/0098-1354(93)E0010-7.

Regis, R.G., Shoemaker, C.A., 2007. A stochastic radial basis function method for the global optimization of expensive functions. INFORMS J. Comput. 19, 497–509. doi:10.1287/ijoc.1060.0182.

Rendle, S., 2010. Factorization machines, in: 2010 IEEE International Conference on Data Mining, pp. 995–1000. doi:10.1109/ICDM.2010.127.

Sahinidis, N.V., 1996. BARON: A general purpose global optimization software package. J. Glob. Optim. 8, 201–205. doi:10.1007/BF00138693.

SECQUOIA, 2026a. Black-Box-for-GDP: benchmark code, experimental data, and analysis. GitHub. URL: https://github.com/SECQUOIA/Black-Box-for-GDP/tree/ea65c5339486b576008df68645ea68d22f7c7213. commit ea65c533; accessed 3 November 2026.

SECQUOIA, 2026b. pd_ising: Lomustine superstructure PharmaPy simulation workflow and data. URL: https://github.com/SECQUOIA/pd_ising/tree/633cf923f07abba88970fa1fa816c4d06769751e/ds-mfg. pull request 28; commit 633cf923f07abba88970fa1fa816c4d06769751e; accessed October 31, 2026.

SECQUOIA, n.d. bbo_via_fmqa: Black-box optimization via factorization machines and quantum annealing. URL: https://github.com/SECQUOIA/bbo_via_fmqa/tree/eb3a20d4b5a3773d17ff31d72a35d51f58e84e56. QHD_Data branch; commit eb3a20d4b5a3773d17ff31d72a35d51f58e84e56; upstream PyTorch FM and BQM code; accessed October 19, 2026.

The Optimization Firm, 2026. BAM python interface documentation: bam-solver 5.0.0. URL: https://pypi.org/project/bam-solver/5.0.0/. Version 5.0.0, released September 11, 2026; executed engine 2026.9.10; accessed October 20, 2026.

Türkay, M., Grossmann, I.E., 1996. Logic-based MINLP algorithms for the optimal synthesis of process networks. Comput. Chem. Eng. 20, 959–978. doi:10.1016/0098-1354(95)00219-7.

Viswanathan, J., Grossmann, I.E., 1990. A combined penalty function and outer-approximation method for MINLP optimization. Computers & Chemical Engineering 14, 769–782. doi:10.1016/0098-1354(90)87085-4.

Wächter, A., Biegler, L.T., 2006. On the implementation of an interior-point filter line-search algorithm for large-scale nonlinear programming. Mathematical Programming 106, 25–57. doi:10.1007/s10107-004-0559-y.

Wu Group, n.d. Quantum Hamiltonian descent: 2D test functions. https://jiaqileng.github.io/quantum-hamiltonian-descent/nonconvex2d.html. Source collection for the 21 adapted finite grids used here; accessed September 29, 2026.